

ObjektinisUzdavinys v2.0

2.0

Sugeneruota Wed May 27 2026 12:16:59 ObjektinisUzdavinys v2.0 Doxygen 1.17.0

Wed May 27 2026 12:16:59

1 Hierarchijos Indeksas	1
1.1 Klasių hierarchija	1
2 Klasės Indeksas	3
2.1 Klasės	3
3 Failo Indeksas	5
3.1 Failai	5
4 Klasės Dokumentacija	7
4.1 deque< T >::const_iterator Klasė	7
4.1.1 Smulkus aprašymas	7
4.2 list< T >::const_iterator Klasė	7
4.2.1 Smulkus aprašymas	7
4.3 string::const_iterator Klasė	7
4.3.1 Smulkus aprašymas	7
4.4 vector< T >::const_iterator Klasė	8
4.4.1 Smulkus aprašymas	8
4.5 deque< T >::const_reverse_iterator Klasė	8
4.5.1 Smulkus aprašymas	8
4.6 list< T >::const_reverse_iterator Klasė	8
4.6.1 Smulkus aprašymas	8
4.7 string::const_reverse_iterator Klasė	8
4.7.1 Smulkus aprašymas	8
4.8 vector< T >::const_reverse_iterator Klasė	9
4.8.1 Smulkus aprašymas	9
4.9 deque< T > Klasė Šablonas	9
4.9.1 Smulkus aprašymas	9
4.10 exception Klasė	9
4.10.1 Smulkus aprašymas	10
4.11 ifstream Klasė	10
4.11.1 Smulkus aprašymas	10
4.12 invalid_argument Klasė	10
4.12.1 Smulkus aprašymas	10
4.13 istream Klasė	10
4.13.1 Smulkus aprašymas	10
4.14 deque< T >::iterator Klasė	10
4.14.1 Smulkus aprašymas	11
4.15 list< T >::iterator Klasė	11
4.15.1 Smulkus aprašymas	11
4.16 string::iterator Klasė	11
4.16.1 Smulkus aprašymas	11
4.17 vector< T >::iterator Klasė	11

4.17.1 Smulkus aprašymas	11
4.18 list< T > Klasė Šablonas	11
4.18.1 Smulkus aprašymas	12
4.19 ofstream Klasė	12
4.19.1 Smulkus aprašymas	12
4.20 ostream Klasė	12
4.20.1 Smulkus aprašymas	12
4.21 deque< T >::reverse_iterator Klasė	13
4.21.1 Smulkus aprašymas	13
4.22 list< T >::reverse_iterator Klasė	13
4.22.1 Smulkus aprašymas	13
4.23 string::reverse_iterator Klasė	13
4.23.1 Smulkus aprašymas	13
4.24 vector< T >::reverse_iterator Klasė	13
4.24.1 Smulkus aprašymas	13
4.25 runtime_error Klasė	14
4.25.1 Smulkus aprašymas	14
4.26 string Klasė	14
4.26.1 Smulkus aprašymas	14
4.27 stringstream Klasė	14
4.27.1 Smulkus aprašymas	14
4.28 Studentas Klasė	15
4.28.1 Smulkus aprašymas	16
4.28.2 Konstruktorius ir Destruktorius Dokumentacija	16
4.28.2.1 Studentas() [1/5]	16
4.28.2.2 Studentas() [2/5]	16
4.28.2.3 Studentas() [3/5]	16
4.28.2.4 ~Studentas()	16
4.28.2.5 Studentas() [4/5]	17
4.28.2.6 Studentas() [5/5]	17
4.28.3 Metodų Dokumentacija	17
4.28.3.1 addPaz()	17
4.28.3.2 clearPaz()	17
4.28.3.3 getEgz()	17
4.28.3.4 getInfo()	17
4.28.3.5 getMed()	18
4.28.3.6 getPaz() [1/2]	18
4.28.3.7 getPaz() [2/2]	18
4.28.3.8 getRez()	18
4.28.3.9 isPazEmpty()	18
4.28.3.10 operator=() [1/2]	18
4.28.3.11 operator=() [2/2]	18

4.28.3.12 readStudent()	19
4.28.3.13 reservePaz()	19
4.28.3.14 setEgz()	19
4.28.3.15 setMed()	19
4.28.3.16 setPaz()	19
4.28.3.17 setRez()	19
4.28.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	19
4.28.4.1 operator<<	19
4.28.4.2 operator>>	20
4.29 vector< T > Klasė Šablonas	20
4.29.1 Smulkus aprašymas	20
4.30 Zmogus Klasė	20
4.30.1 Smulkus aprašymas	21
4.30.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	22
4.30.2.1 Zmogus() [1/2]	22
4.30.2.2 Zmogus() [2/2]	22
4.30.2.3 ~Zmogus()	22
4.30.3 Metodų Dokumentacija	22
4.30.3.1 getInfo()	22
4.30.3.2 getPavarde()	23
4.30.3.3 getVardas()	23
4.30.3.4 setPavarde()	23
4.30.3.5 setVardas()	23
4.30.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	24
4.30.4.1 operator<<	24
4.30.5 Atributų Dokumentacija	24
4.30.5.1 pavarde_	24
4.30.5.2 vardas_	24
5 Failo Dokumentacija	25
5.1 duomenu_valdymas.cpp	25
5.2 input.cpp	30
5.3 mat_funkcijos.cpp	31
5.4 menu.cpp	32
5.5 output.cpp	33
5.6 Studentas.cpp	34
5.7 testavimas.cpp	36
5.8 Zmogus.cpp	41
5.9 duomenu_valdymas.h	41
5.10 input.h	41
5.11 Mat_funkcijos.h	42
5.12 menu.h	42

5.13 output.h	42
5.14 Studentas.h	42
5.15 testavimas.h	43
5.16 Zmogus.h	43
5.17 objekt1uzd.cpp	44
5.18 test_studentas.cpp	45
Rodyklė	51

skyrius 1

Hierarchijos Indeksas

1.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

deque< T >const_iterator	7
list< T >const_iterator	7
string::const_iterator	7
vector< T >const_iterator	8
deque< T >const_reverse_iterator	8
list< T >const_reverse_iterator	8
string::const_reverse_iterator	8
vector< T >const_reverse_iterator	9
deque< T >	9
exception	9
ifstream	10
invalid_argument	10
istream	10
deque< T >iterator	10
list< T >iterator	11
string::iterator	11
vector< T >iterator	11
list< T >	11
ofstream	12
ostream	12
deque< T >reverse_iterator	13
list< T >reverse_iterator	13
string::reverse_iterator	13
vector< T >reverse_iterator	13
runtime_error	14
string	14
stringstream	14
vector< T >	20
Zmogus	20
Studentas	15

skyrius 2

Klasės Indeksas

2.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

deque< T >::const_iterator	
STL iterator class	7
list< T >::const_iterator	
STL iterator class	7
string::const_iterator	
STL iterator class	7
vector< T >::const_iterator	
STL iterator class	8
deque< T >::const_reverse_iterator	
STL iterator class	8
list< T >::const_reverse_iterator	
STL iterator class	8
string::const_reverse_iterator	
STL iterator class	8
vector< T >::const_reverse_iterator	
STL iterator class	9
deque< T >	
STL class	9
exception	
STL class	9
ifstream	
STL class	10
invalid_argument	
STL class	10
istream	
STL class	10
deque< T >::iterator	
STL iterator class	10
list< T >::iterator	
STL iterator class	11
string::iterator	
STL iterator class	11
vector< T >::iterator	
STL iterator class	11
list< T >	
STL class	11

ofstream		
STL class	12	
ostream		
STL class	12	
deque< T >::reverse_iterator		
STL iterator class	13	
list< T >::reverse_iterator		
STL iterator class	13	
string::reverse_iterator		
STL iterator class	13	
vector< T >::reverse_iterator		
STL iterator class	13	
runtime_error		
STL class	14	
string		
STL class	14	
stringstream		
STL class	14	
Studentas		15
vector< T >		
STL class	20	
Zmogus		
Abstrakti bazinė klasė bendram žmogaus aprašymui	20	

skyrius 3

Failo Indeksas

3.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

objekt1uzd.cpp	44
test_studentas.cpp	45
extra_cpp/duomenu_valdymas.cpp	25
extra_cpp/input.cpp	30
extra_cpp/mat_funkcijos.cpp	31
extra_cpp/menu.cpp	32
extra_cpp/output.cpp	33
extra_cpp/Studentas.cpp	34
extra_cpp/testavimas.cpp	36
extra_cpp/Zmogus.cpp	41
header_files/duomenu_valdymas.h	41
header_files/input.h	41
header_files/Mat_funkcijos.h	42
header_files/menu.h	42
header_files/output.h	42
header_files/Studentas.h	42
header_files/testavimas.h	43
header_files/Zmogus.h	43

skyrius 4

Klasės Dokumentacija

4.1 `deque< T >::const_iterator` Klasė

STL iterator class.

4.1.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.2 `list< T >::const_iterator` Klasė

STL iterator class.

4.2.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.3 `string::const_iterator` Klasė

STL iterator class.

4.3.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.4 `vector< T >::const_iterator` Klasė

STL iterator class.

4.4.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.5 `deque< T >::const_reverse_iterator` Klasė

STL iterator class.

4.5.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.6 `list< T >::const_reverse_iterator` Klasė

STL iterator class.

4.6.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.7 `string::const_reverse_iterator` Klasė

STL iterator class.

4.7.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.8 `vector< T >::const_reverse_iterator` Klasė

STL iterator class.

4.8.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.9 `deque< T >` Klasė Šablonas

STL class.

Klasės

- class `iterator`
STL iterator class.
- class `const_iterator`
STL iterator class.
- class `reverse_iterator`
STL iterator class.
- class `const_reverse_iterator`
STL iterator class.

Vieši Atributai

- `T elements`
STL member.

4.9.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.10 `exception` Klasė

STL class.

4.10.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.11 ifstream Klasė

STL class.

4.11.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.12 invalid_argument Klasė

STL class.

4.12.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.13 istream Klasė

STL class.

4.13.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.14 deque< T >::iterator Klasė

STL iterator class.

4.14.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.15 list< T >::iterator Klasė

STL iterator class.

4.15.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.16 string::iterator Klasė

STL iterator class.

4.16.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.17 vector< T >::iterator Klasė

STL iterator class.

4.17.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.18 list< T > Klasė Šablonas

STL class.

Klasės

- class `iterator`
STL iterator class.
- class `const_iterator`
STL iterator class.
- class `reverse_iterator`
STL iterator class.
- class `const_reverse_iterator`
STL iterator class.

Vieši Atributai

- `T elements`
STL member.

4.18.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.19 ofstream Klasė

STL class.

4.19.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.20 ostream Klasė

STL class.

4.20.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.21 deque< T >::reverse_iterator Klasė

STL iterator class.

4.21.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.22 list< T >::reverse_iterator Klasė

STL iterator class.

4.22.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.23 string::reverse_iterator Klasė

STL iterator class.

4.23.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.24 vector< T >::reverse_iterator Klasė

STL iterator class.

4.24.1 Smulkus aprašymas

STL iterator class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.25 runtime_error Klasė

STL class.

4.25.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.26 string Klasė

STL class.

Klasės

- class [iterator](#)
STL iterator class.
- class [const_iterator](#)
STL iterator class.
- class [reverse_iterator](#)
STL iterator class.
- class [const_reverse_iterator](#)
STL iterator class.

4.26.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.27 stringstream Klasė

STL class.

4.27.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.28 Studentas Klasė

Paveldimumo diagrama Studentas:



Vieši Metodai

- `Studentas` (`string` v, `string` p, `int` e)
- `Studentas` (`std::istream` &is)
- `Studentas` (`const Studentas` &other)
- `Studentas` & `operator=` (`const Studentas` &other)
- `Studentas` (`Studentas` &&other) `noexcept`
- `Studentas` & `operator=` (`Studentas` &&other) `noexcept`
- `const vector`< `int` > & `getPaz` () `const`
- `vector`< `int` > & `getPaz` ()
- `int` `getEgz` () `const`
- `double` `getRez` () `const`
- `double` `getMed` () `const`
- `void` `setEgz` (`int` e)
- `void` `setRez` (`double` r)
- `void` `setMed` (`double` m)
- `void` `addPaz` (`int` p)
- `void` `clearPaz` ()
- `void` `reservePaz` (`size_t` n)
- `bool` `isPazEmpty` () `const`
- `void` `setPaz` (`const vector`< `int` > &p)
- `std::istream` & `readStudent` (`std::istream` &is)
- `virtual string` `getInfo` () `const` override

Grąžina žmogaus informaciją Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Vieši Metodai inherited from `Zmogus`

- `Zmogus` (`const string` &v, `const string` &p)
Parametrizuotas konstruktorius.
- `Zmogus` ()
Parametrizuotas konstruktorius su numatytosiomis reikšmėmis.
- `virtual ~Zmogus` ()
Virtualus destruktorius - reikalingas polimorfizmui.
- `string` `getVardas` () `const`
- `string` `getPavarde` () `const`
- `void` `setVardas` (`const string` &v)
Nustato žmogaus vardą
- `void` `setPavarde` (`const string` &p)
Nustato žmogaus pavardę

Draugai

- `std::ostream & operator<<` (`std::ostream &os`, `const Studentas &s`)
- `std::istream & operator>>` (`std::istream &is`, `Studentas &s`)

Additional Inherited Members

Apsaugoti Atributai inherited from Zmogus

- `string vardas_`
- `string pavarde_`

4.28.1 Smulkus aprašymas

Apibrėžimas failo [Studentas.h](#) eilutėje 15.

4.28.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.28.2.1 Studentas() [1/5]

```
Studentas::Studentas () [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 30.

4.28.2.2 Studentas() [2/5]

```
Studentas::Studentas (  
    string v,  
    string p,  
    int e) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 33.

4.28.2.3 Studentas() [3/5]

```
Studentas::Studentas (  
    std::istream & is)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 8.

4.28.2.4 ~Studentas()

```
Studentas::~Studentas ()
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 15.

4.28.2.5 Studentas() [4/5]

```
Studentas::Studentas (
    const Studentas & other)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 20.

4.28.2.6 Studentas() [5/5]

```
Studentas::Studentas (
    Studentas && other) [noexcept]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 39.

4.28.3 Metodų Dokumentacija

4.28.3.1 addPaz()

```
void Studentas::addPaz (
    int p) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 68.

4.28.3.2 clearPaz()

```
void Studentas::clearPaz () [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 69.

4.28.3.3 getEgz()

```
int Studentas::getEgz () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 58.

4.28.3.4 getInfo()

```
string Studentas::getInfo () const [override], [virtual]
```

Gražina žmogaus informaciją Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Realizuoja [Zmogus](#).

Apibrėžimas failo [Studentas.cpp](#) eilutėje 68.

Ryšiai [Zmogus::getPavarde\(\)](#), ir [Zmogus::getVardas\(\)](#).

4.28.3.5 getMed()

```
double Studentas::getMed () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 60.

4.28.3.6 getPaz() [1/2]

```
vector< int > & Studentas::getPaz () [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 57.

4.28.3.7 getPaz() [2/2]

```
const vector< int > & Studentas::getPaz () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 56.

4.28.3.8 getRez()

```
double Studentas::getRez () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 59.

4.28.3.9 isPazEmpty()

```
bool Studentas::isPazEmpty () const [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 71.

4.28.3.10 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 26.

4.28.3.11 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 50.

4.28.3.12 readStudent()

```
std::istream & Studentas::readStudent (
    std::istream & is)
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 91.

4.28.3.13 reservePaz()

```
void Studentas::reservePaz (
    size_t n) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 70.

4.28.3.14 setEgz()

```
void Studentas::setEgz (
    int e) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 63.

4.28.3.15 setMed()

```
void Studentas::setMed (
    double m) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 65.

4.28.3.16 setPaz()

```
void Studentas::setPaz (
    const vector< int > & p) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 72.

4.28.3.17 setRez()

```
void Studentas::setRez (
    double r) [inline]
```

Apibrėžimas failo [Studentas.h](#) eilutėje 64.

4.28.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.28.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s) [friend]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 111.

4.28.4.2 operator>>

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s) [friend]
```

Apibrėžimas failo [Studentas.cpp](#) eilutėje 128.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- header_files/Studentas.h
- extra_cpp/Studentas.cpp

4.29 vector< T > Klasė Šablonas

STL class.

Klasės

- class [iterator](#)
STL iterator class.
- class [const_iterator](#)
STL iterator class.
- class [reverse_iterator](#)
STL iterator class.
- class [const_reverse_iterator](#)
STL iterator class.

Vieši Atributai

- **T elements**
STL member.

4.29.1 Smulkus aprašymas

STL class.

Dokumentacija šiai klasei sugeneruota iš šių failų:

4.30 Zmogus Klasė

Abstrakti bazinė klasė bendram žmogaus aprašymui.

```
#include <Zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- `Zmogus` (`const string &v`, `const string &p`)
Parametrizuotas konstruktorius.
- `Zmogus` ()
Parametrizuotas konstruktorius su numatytosiomis reikšmėmis.
- `virtual ~Zmogus` ()
Virtualus destruktorius - reikalingas polimorfizmui.
- `string getVardas` () `const`
- `string getPavarde` () `const`
- `void setVardas` (`const string &v`)
Nustato žmogaus vardą
- `void setPavarde` (`const string &p`)
Nustato žmogaus pavardę
- `virtual string getInfo` () `const` =0
Grąžina žmogaus informaciją Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Apsaugoti Metodai

- `virtual void paskaiciuoti` ()=0
Pagalbinis metodas žmogaus duomenims paskaiciuoti Abstraktus metodas - turi būti implementuotas išvestinėse klasėse.

Apsaugoti Atributai

- `string vardas_`
- `string pavarde_`

Draugai

- `std::ostream & operator<<` (`std::ostream &os`, `const Zmogus &z`)
Išvesties operatorius - spausdina žmogaus duomenis.

4.30.1 Smulkus aprašymas

Abstrakti bazinė klasė bendram žmogaus aprašymui.

Ši klasė nuo v1.5 versijos aprašo bendrą žmogaus duomenis (vardas, pavardė). Yra abstrakti - jos objektų kurti negalima, tik iš jos išvestinės klasės.

Paveldi iš šios klasės: `Studentas`

Apibrėžimas failo `Zmogus.h` eilutėje 17.

4.30.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

4.30.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus (
    const string & v,
    const string & p) [inline]
```

Parametrizuotas konstruktorius.

Parametrai

<i>v</i>	Žmogaus vardas
<i>p</i>	Žmogaus pavardė

Apibrėžimas failo [Zmogus.h](#) eilutėje 34.

Naudojamas [getInfo\(\)](#), ir [operator<<](#).

4.30.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus () [inline]
```

Parametrizuotas konstruktorius su numatytosiomis reikšmėmis.

Apibrėžimas failo [Zmogus.h](#) eilutėje 40.

4.30.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [inline], [virtual]
```

Virtualus destruktoriaus - reikalingas polimorfizmui.

Apibrėžimas failo [Zmogus.h](#) eilutėje 45.

4.30.3 Metodų Dokumentacija

4.30.3.1 getInfo()

```
virtual string Zmogus::getInfo () const [pure virtual]
```

Gražina žmogaus informaciją Šis metodas turi būti implementuotas visose išvestinėse klasėse.

Realizuota [Studentas](#).

Ryšiai [Zmogus\(\)](#).

4.30.3.2 getPavarde()

```
string Zmogus::getPavarde () const [inline]
```

Gražina

Žmogaus pavarde

Apibrėžimas failo [Zmogus.h](#) eilutėje 56.

Naudojamas [Studentas::getInfo\(\)](#).

4.30.3.3 getVardas()

```
string Zmogus::getVardas () const [inline]
```

Gražina

Žmogaus vardą

Apibrėžimas failo [Zmogus.h](#) eilutėje 51.

Naudojamas [Studentas::getInfo\(\)](#).

4.30.3.4 setPavarde()

```
void Zmogus::setPavarde (  
    const string & p) [inline]
```

Nustato žmogaus pavarde

Apibrėžimas failo [Zmogus.h](#) eilutėje 67.

4.30.3.5 setVardas()

```
void Zmogus::setVardas (  
    const string & v) [inline]
```

Nustato žmogaus vardą

Apibrėžimas failo [Zmogus.h](#) eilutėje 62.

4.30.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

4.30.4.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Zmogus & z) [friend]
```

Išvesties operatorius - spausdina žmogaus duomenis.

Parametrai

os	Išvesties srautas
z	Žmogaus objektas

Gražina

Išvesties srautas

Apibrėžimas failo [Zmogus.cpp](#) eilutėje 12.

Ryšiai [operator<<](#), ir [Zmogus\(\)](#).

Naudojamas [operator<<](#).

4.30.5 Atributų Dokumentacija

4.30.5.1 pavarde_

```
string Zmogus::pavarde_ [protected]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 20.

4.30.5.2 vardas_

```
string Zmogus::vardas_ [protected]
```

Apibrėžimas failo [Zmogus.h](#) eilutėje 19.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [header_files/Zmogus.h](#)

skyrius 5

Failo Dokumentacija

5.1 duomenu_valdymas.cpp

```
00001 #include <vector>
00002 #include <iostream>
00003 #include <string>
00004 #include <algorithm>
00005 #include <cctype>
00006 #include <limits>
00007 #include <random>
00008 #include <ctime>
00009 #include <fstream>
00010 #include <iomanip>
00011 #include <chrono>
00012 #include <stdexcept>
00013
00014 #include "../header_files/duomenu_valdymas.h"
00015 #include "../header_files/mat_funkcijos.h"
00016 #include "../header_files/output.h"
00017
00018 using std::cin;
00019 using std::cout;
00020 using std::string;
00021 using std::vector;
00022 using std::numeric_limits;
00023 using std::streamsize;
00024 using std::ifstream;
00025 using std::ofstream;
00026 using std::left;
00027 using std::setw;
00028 using std::fixed;
00029 using std::setprecision;
00030 using std::sort;
00031 using std::chrono::high_resolution_clock;
00032 using std::chrono::duration;
00033 using std::runtime_error;
00034 using std::exception;
00035
00036
00037
00038
00039 // Atsitiktiniu skaiciu generavimas
00040 void generuotiPaz(vector<Studentas>& grupe) {
00041     srand(time(0));
00042
00043     bool testiStudenta = true;
00044
00045     while (testiStudenta) {
00046         Studentas A;
00047
00048         bool vardasGeras = false;
00049         while (!vardasGeras) {
00050             cout << "Iveskite varda ir pavarde: ";
00051             cin >> std::ws; string v, p; cin >> v >> p; A.setVardas(v); A.setPavarde(p);
00052
00053             if (!arTikRaides(A.getVardas()) || !arTikRaides(A.getPavarde())) {
00054                 cout << "Klaida! Vardas ir pavarde turi buti sudaryti tik is raidziu!\n";
00055                 cin.clear();
00056             }
00057             else {
```

```

00058         vardasGeras = true;
00059     }
00060 }
00061
00062 cout << string(80, '-') << "\n";
00063
00064 // Automatiskai generuojamas atsitiktinis pazymiu kiekis (3-10)
00065 int n = rand() % 8 + 3;
00066 A.reservePaz(n); // Atminties rezervacija pazymiams
00067
00068 // Automatiskai generuojami pazymiai
00069 cout << "Sugeneruota " << n << " pazymiu: ";
00070 for (int i = 0; i < n; i++) {
00071     int temp = generuotiPazymi();
00072     A.addPaz(temp);
00073     cout << temp << " ";
00074 }
00075 cout << "\n";
00076
00077 // Automatiskai generuojamas egzaminas
00078 A.setEgz(generuotiEgzamina());
00079 cout << "Egzamino ivertinimas: " << A.getEgz() << "\n";
00080 cout << string(80, '-') << "\n";
00081
00082 // Vidurkio skaiciavimas
00083 if (!A.isPazEmpty()) {
00084     double vid = vidurkis(A.getPaz());
00085     A.setRez(galutinisBalas(vid, A.getEgz()));
00086     // Medianos skaiciavimas
00087     double med = mediana(A.getPaz());
00088     A.setMed(galutinisBalas(med, A.getEgz()));
00089 }
00090 else {
00091     A.setRez(A.getEgz() * 0.6);
00092     A.setMed(A.getEgz() * 0.6);
00093 }
00094
00095 grupe.push_back(A);
00096
00097 // Klausimas ar testi
00098 char atsakymas;
00099 bool atsakymasTeisingas = false;
00100 while (!atsakymasTeisingas) {
00101     cout << "Ar norite generuoti dar viena studenta? (T/N): ";
00102     cin >> atsakymas;
00103     if (atsakymas == 'T' || atsakymas == 't') {
00104         testiStudenta = true;
00105         atsakymasTeisingas = true;
00106         cout << string(80, '-') << "\n";
00107     }
00108     else if (atsakymas == 'N' || atsakymas == 'n') {
00109         testiStudenta = false;
00110         atsakymasTeisingas = true;
00111     }
00112     else {
00113         cout << "Klaida! Iveskite T arba N!\n";
00114         cin.clear();
00115         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00116     }
00117 }
00118 }
00119 }
00120
00121 void generuotiVardIrPav(vector<Studentas>& grupe) {
00122     srand(time(0));
00123
00124     try {
00125         // Nuskaitome failus
00126         vector<string> vyruVard, vyruPav, motVard, motPav;
00127         ifstream vvard("../vpv/vvard.txt");
00128         ifstream vpav("../vpv/vpav.txt");
00129         ifstream mvard("../vpv/mvard.txt");
00130         ifstream mpav("../vpv/mpav.txt");
00131
00132         // Patikrinimas ar failai atsidare
00133         if (!vvard.is_open() || !vpav.is_open() || !mvard.is_open() || !mpav.is_open()) {
00134             throw runtime_error("Nepavyko atidaryti vieno ar daugiau failu su vardais ir
pavardemis!");
00135         }
00136
00137         // Nuskaitomi visi vardai ir pavardes
00138         string eilute;
00139         while (vvard >> eilute) vyruVard.push_back(eilute);
00140         while (vpav >> eilute) vyruPav.push_back(eilute);
00141         while (mvard >> eilute) motVard.push_back(eilute);
00142         while (mpav >> eilute) motPav.push_back(eilute);
00143

```



```

00144     vvard.close();
00145     vpav.close();
00146     mvard.close();
00147     mpav.close();
00148
00149     // Patikrinama ar failai ne tusti
00150     if (vyruVard.empty() || vyruPav.empty() || motVard.empty() || motPav.empty()) {
00151         throw runtime_error("Vienas ar daugiau failu yra tusti!");
00152     }
00153
00154     bool testiStudenta = true;
00155
00156     while (testiStudenta) {
00157         Studentas A;
00158
00159         // Atsitiktinai parenka lyti (0 - vyras, 1 - moteris)
00160         int lytis = rand() % 2;
00161
00162         // Generuojamas vardas ir pavarde pagal lyti
00163         if (lytis == 0) {
00164             int vardIndex = rand() % vyruVard.size();
00165             int pavIndex = rand() % vyruPav.size();
00166             A.setVardas(vyruVard[vardIndex]);
00167             A.setPavarde(vyruPav[pavIndex]);
00168         }
00169         else {
00170             int vardIndex = rand() % motVard.size();
00171             int pavIndex = rand() % motPav.size();
00172             A.setVardas(motVard[vardIndex]);
00173             A.setPavarde(motPav[pavIndex]);
00174         }
00175
00176         cout << "Sugeneruotas vardas ir pavarde: " << A.getVardas() << " " << A.getPavarde() << "\n";
00177         cout << string(80, '-') << "\n";
00178
00179         // Automatiskai generuojamas atsitiktinis pazymiu kiekis (3-10)
00180         int n = rand() % 8 + 3;
00181         A.reservePaz(n); // Atminties rezervacija pazymiams
00182
00183         // Automatiskai sugeneruojami pazymiai
00184         cout << "Sugeneruota " << n << " pazymiu: ";
00185         for (int i = 0; i < n; i++) {
00186             int temp = generuotiPazymi();
00187             A.addPaz(temp);
00188             cout << temp << " ";
00189         }
00190         cout << "\n";
00191
00192         // Automatiskai sugeneruojamas egzaminas
00193         A.setEgz(generuotiEgzamina());
00194         cout << "Egzamino ivertinimas: " << A.getEgz() << "\n";
00195         cout << string(80, '-') << "\n";
00196
00197         // Vidurkio skaiciavimas
00198         if (!A.isPazEmpty()) {
00199             double vid = vidurkis(A.getPaz());
00200             A.setRez(galutinisBalas(vid, A.getEgz()));
00201             // Medianos skaiciavimas
00202             double med = mediana(A.getPaz());
00203             A.setMed(galutinisBalas(med, A.getEgz()));
00204         }
00205         else {
00206             A.setRez(A.getEgz() * 0.6);
00207             A.setMed(A.getEgz() * 0.6);
00208         }
00209
00210         grupe.push_back(A);
00211
00212         // Klausimas ar testi
00213         char atsakymas;
00214         bool atsakymasTeisingas = false;
00215         while (!atsakymasTeisingas) {
00216             cout << "Ar norite generuoti dar viena studenta? (T/N): ";
00217             cin >> atsakymas;
00218             if (atsakymas == 'T' || atsakymas == 't') {
00219                 testiStudenta = true;
00220                 atsakymasTeisingas = true;
00221                 cout << string(80, '-') << "\n";
00222             }
00223             else if (atsakymas == 'N' || atsakymas == 'n') {
00224                 testiStudenta = false;
00225                 atsakymasTeisingas = true;
00226             }
00227             else {
00228                 cout << "Klaida! Iveskite T arba N!\n";
00229                 cin.clear();
00230                 cin.ignore(numeric_limits<streamsize>::max(), '\n');

```

```

00231     }
00232     }
00233 }
00234 }
00235 catch (const runtime_error& e) {
00236     cout << "Klaida generuojant studentus: " << e.what() << "\n";
00237 }
00238 catch (const exception& e) {
00239     cout << "Netiketa klaida: " << e.what() << "\n";
00240 }
00241 }
00242
00243 void skaitytiIsFailo(vector<Studentas>& grupe) {
00244     try {
00245         string failoPavadinimas;
00246         cout << "Iveskite failo pavadinima: ";
00247         cin >> failoPavadinimas;
00248
00249         // Pridedamas testuojamas katalogas
00250         string kelias = "..\\studentai_test\\" + failoPavadinimas;
00251
00252         ifstream failas(kelias);
00253         if (!failas.is_open()) {
00254             throw runtime_error("Nepavyko atidaryti failo: " + kelias);
00255         }
00256
00257         // Praleidžiama antraste
00258         string eilute;
00259         getline(failas, eilute);
00260
00261         // Skaitomi studentai
00262         while (failas >> eilute) {
00263             Studentas A;
00264             A.setVardas(eilute);
00265             failas >> A.getPavarde();
00266
00267             vector<int> paz;
00268             paz.reserve(15); // Rezervuojama talpa (~15 pazymiu dazniausiai pakanka)
00269             int sk;
00270             while (failas >> sk) {
00271                 paz.push_back(sk);
00272                 if (failas.peek() == '\n') break;
00273             }
00274
00275             if (!paz.empty()) {
00276                 A.setEgz(paz.back());
00277                 paz.pop_back();
00278                 A.setPaz(paz);
00279                 A.setRez(galutinisBalas(vidurkis(A.getPaz()), A.getEgz()));
00280                 A.setMed(galutinisBalas(mediana(A.getPaz()), A.getEgz()));
00281                 grupe.push_back(A);
00282             }
00283             failas.ignore(numeric_limits<streamsize>::max(), '\n');
00284         }
00285
00286         failas.close();
00287
00288         if (grupe.empty()) {
00289             throw runtime_error("Failas yra tuscias arba neteisingai formatuotas!");
00290         }
00291
00292         cout << "Nuskaityta " << grupe.size() << " studentu!\n";
00293
00294         // Klausinama kur isvesti rezultatus
00295         if (!grupe.empty()) {
00296             cout << string(80, '-') << "\n";
00297             cout << "Pasirinkite isvesties buda:\n";
00298             cout << "1. Isvesti i terminala\n";
00299             cout << "2. Irasyti i faila\n";
00300             int isvestiesPasirinkimas;
00301             while (!(cin >> isvestiesPasirinkimas) || (isvestiesPasirinkimas != 1 &&
isvestiesPasirinkimas != 2)) {
00302                 cout << "Klaida! Pasirinkite 1 arba 2: ";
00303                 cin.clear();
00304                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00305             }
00306             if (isvestiesPasirinkimas == 1) {
00307                 outputas(grupe);
00308             }
00309             else {
00310                 rasytiIFaila(grupe);
00311             }
00312         }
00313     }
00314     catch (const runtime_error& e) {
00315         cout << "Klaida skaitant is failo: " << e.what() << "\n";
00316     }

```

```

00317     catch (const exception& e) {
00318         cout << "Netiketa klaida: " << e.what() << "\n";
00319     }
00320 }
00321
00322 void rasytIFaila(vector<Studentas>& grupe) {
00323     if (grupe.empty()) {
00324         cout << "Nera studentu duomenu!\n";
00325         return;
00326     }
00327
00328     // Klausima kaip rusiuoti
00329     cout << string(80, '-') << "\n";
00330     cout << "Pasirinkite rusiavimo buda:\n";
00331     cout << "1. Pagal varda (A-Z)\n";
00332     cout << "2. Pagal pavarde (A-Z)\n";
00333     cout << "3. Pagal galutini bala (vidurki) - didejimo tvarka\n";
00334     cout << "4. Pagal galutini bala (mediana) - didejimo tvarka\n";
00335
00336     int pasirinkimas;
00337     while (!(cin >> pasirinkimas) || pasirinkimas < 1 || pasirinkimas > 4) {
00338         cout << "Klaida! Iveskite skaiciu nuo 1 iki 4: ";
00339         cin.clear();
00340         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00341     }
00342
00343     cout << "Pasirinktas rusiavimas: " << pasirinkimas << "\n";
00344
00345     // Pradedamas rusiavimo laiko matavimas
00346     auto pradzia = high_resolution_clock::now();
00347
00348     // Rusiuojama pagal pasirinkima
00349     switch (pasirinkimas) {
00350     case 1:
00351         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00352             return a.getVardas() < b.getVardas();
00353         });
00354         break;
00355     case 2:
00356         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00357             return a.getPavarde() < b.getPavarde();
00358         });
00359         break;
00360     case 3:
00361         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00362             return a.getRez() < b.getRez();
00363         });
00364         break;
00365     case 4:
00366         sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00367             return a.getMed() < b.getMed();
00368         });
00369         break;
00370     default:
00371         cout << "Klaida! Neteisingas pasirinkimas. Rodoma be rusiavimo.\n";
00372         break;
00373     }
00374
00375     // Baigiamas rusiavimo laiko matavimas
00376     auto pabaiga = high_resolution_clock::now();
00377     duration<double> trukme = pabaiga - pradzia;
00378
00379     // Klausima failo pavadinimo
00380     string isvestiesFailas;
00381     cout << string(80, '-') << "\n";
00382     cout << "Iveskite isvesties failo pavadinima: ";
00383     cin >> isvestiesFailas;
00384
00385     // Sukuriamas kelias i testuojama kataloga
00386     string kelias = "..\\studentai_test\\" + isvestiesFailas;
00387
00388     ofstream failas(kelias);
00389     if (!failas.is_open()) {
00390         cout << "Klaida! Nepavyko sukurti failo: " << kelias << "\n";
00391         return;
00392     }
00393
00394     // Antrastes eilute
00395     failas << left << setw(20) << "Vardas" << setw(20) << "Pavarde"
00396           << setw(20) << "Galutinis (Vid.)" << setw(20) << "Galutinis (Med.)\n";
00397     failas << string(80, '-') << "\n";
00398
00399     // Studentu duomenys
00400     for (const auto& A : grupe) {
00401         failas << left << setw(20) << A.getVardas() << setw(20) << A.getPavarde()
00402               << setw(20) << fixed << setprecision(2) << A.getRez()
00403               << setw(20) << fixed << setprecision(2) << A.getMed() << "\n";

```

```

00404     }
00405
00406     failas.close();
00407
00408     cout << string(80, '-') << "\n";
00409     cout << "Duomenys sekmingai irasyti i faila: " << kelias << "\n";
00410     cout << "Ivykdymo laikas: " << fixed << setprecision(7) << trukme.count() << " s\n";
00411 }

```

5.2 input.cpp

```

00001 #include <iostream>
00002 #include <limits>
00003
00004 #include "../header_files/input.h"
00005 #include "../header_files/mat_funkcijos.h"
00006
00007 using std::cin;
00008 using std::cout;
00009 using std::vector;
00010 using std::string;
00011 using std::numeric_limits;
00012 using std::streamsize;
00013
00014 void inputas(vector<Studentas>& grupe) {
00015     bool testiStudenta = true;
00016
00017     while (testiStudenta) {
00018         Studentas A;
00019
00020         bool vardasGeras = false;
00021         while (!vardasGeras) {
00022             cout << "Iveskite varda ir pavarde: ";
00023             cin >> std::ws; string v, p; cin >> v >> p; A.setVardas(v); A.setPavarde(p);
00024
00025             if (!arTikRaides(A.getVardas()) || !arTikRaides(A.getPavarde())) {
00026                 cout << "Klaida! Vardas ir pavarde turi buti sudaryti tik is raidziu!\n";
00027                 cin.clear();
00028             }
00029             else {
00030                 vardasGeras = true;
00031             }
00032         }
00033
00034         cout << string(80, '-') << "\n";
00035         cout << "Iveskite semestro ivertinimus (0-10). Iveskite -1 kad baigtumete: \n";
00036         A.reservePaz(15); // Rezervuojama vieta pazymiams (sumazina atminties reallokacijas)
00037         int temp;
00038         while (true) {
00039             cout << "Iveskite pazymi (arba -1 kad baigtumete): ";
00040             if (!(cin >> temp)) {
00041                 cout << "Klaida! Iveskite skaiciu!\n";
00042                 cin.clear();
00043                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00044             }
00045             else if (temp == -1) {
00046                 break;
00047             }
00048             else if (temp < 0 || temp > 10) {
00049                 cout << "Klaida! Pazymys turi buti nuo 0 iki 10!\n";
00050             }
00051             else {
00052                 A.addPaz(temp);
00053             }
00054         }
00055
00056         cout << string(80, '-') << "\n";
00057
00058         bool egzaminasTeisingas = false;
00059         while (!egzaminasTeisingas) {
00060             cout << "Iveskite egzamina (0-10): ";
00061             int e; if (!(cin >> e)) {
00062                 cout << "Klaida! Iveskite skaiciu!\n";
00063                 cin.clear();
00064                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00065             }
00066             else if (e < 0 || e > 10) {
00067                 cout << "Klaida! Egzaminas turi buti nuo 0 iki 10!\n";
00068             }
00069             else {
00070                 A.setEgz(e); egzaminasTeisingas = true;
00071             }
00072         }

```

```

00073         cout << string(80, '-') << "\n";
00074
00075         // Vidurkio skaiciavimas
00076         if (!A.isPazEmpty()) {
00077             double vid = vidurkis(A.getPaz());
00078             A.setRez(galutinisBalas(vid, A.getEgz()));
00079             // Medianos skaiciavimas
00080             double med = mediana(A.getPaz());
00081             A.setMed(galutinisBalas(med, A.getEgz()));
00082         }
00083         else {
00084             A.setRez(A.getEgz() * 0.6);
00085             A.setMed(A.getEgz() * 0.6);
00086         }
00087
00088         grupe.push_back(A);
00089         A.clearPaz();
00090
00091         // Klausimas ar testi rankini generavima
00092         char atsakymas;
00093         bool atsakymasTeisingas = false;
00094         while (!atsakymasTeisingas) {
00095             cout << "Ar norite ivesti dar viena studenta? (T/N): ";
00096             cin >> atsakymas;
00097             if (atsakymas == 'T' || atsakymas == 't') {
00098                 testiStudenta = true;
00099                 atsakymasTeisingas = true;
00100                 cout << string(80, '-') << "\n";
00101             }
00102             else if (atsakymas == 'N' || atsakymas == 'n') {
00103                 testiStudenta = false;
00104                 atsakymasTeisingas = true;
00105             }
00106             else {
00107                 cout << "Klaida! Iveskite T arba N!\n";
00108                 cin.clear();
00109                 cin.ignore(numeric_limits<streamsize>::max(), '\n');
00110             }
00111         }
00112     }
00113 }
00114

```

5.3 mat_funkcijos.cpp

```

00001 #include "../header_files/mat_funkcijos.h"
00002 #include <algorithm>
00003 #include <cctype>
00004 #include <ctime>
00005
00006 using std::sort;
00007
00008 // Funkcija patikrinti ar vardas/pavarde turi tik raides, neleidzia jokiu simboliu
00009 bool arTikRaides(const string& str) {
00010     if (str.empty()) return false;
00011     for (char c : str) {
00012         if (!std::isalpha(c)) {
00013             return false;
00014         }
00015     }
00016     return true;
00017 }
00018
00019 // Medianos funkcija
00020 double mediana(vector<int> paz) {
00021     if (paz.empty()) return 0.0;
00022
00023     vector<int> temp = paz;
00024     sort(temp.begin(), temp.end());
00025     int n = temp.size();
00026     if (n % 2 == 0) {
00027         return (temp[n / 2 - 1] + temp[n / 2]) / 2.0;
00028     }
00029     else {
00030         return temp[n / 2];
00031     }
00032 }
00033
00034 // Vidurkio skaiciavimas is pazymiu vektoriaus
00035 double vidurkis(const vector<int>& paz) {
00036     if (paz.empty()) return 0.0;
00037     int sum = 0;
00038     for (int p : paz) {

```

```

00039         sum += p;
00040     }
00041     return sum * 1.0 / paz.size();
00042 }
00043
00044 // Galutinio balo skaičiavimas pagal vidurki/mediana ir egzamina
00045 double galutinisBalas(double vidMed, int egz) {
00046     return vidMed * 0.4 + egz * 0.6;
00047 }
00048
00049 // Generuoti atsitiktini pazymi nuo 0 iki 10
00050 int generuotiPazymi() {
00051     return rand() % 11;
00052 }
00053
00054 // Generuoti atsitiktini egzamina nuo 0 iki 10
00055 int generuotiEgzamina() {
00056     return rand() % 11;
00057 }

```

5.4 menu.cpp

```

00001 #include <vector>
00002 #include <iostream>
00003 #include <string>
00004 #include <stdexcept>
00005
00006 #include "../header_files/menu.h"
00007 #include "../header_files/studentas.h"
00008 #include "../header_files/input.h"
00009 #include "../header_files/output.h"
00010 #include "../header_files/duomenu_valdymas.h"
00011 #include "../header_files/testavimas.h"
00012
00013 using std::cout;
00014 using std::cin;
00015 using std::string;
00016 using std::vector;
00017 using std::invalid_argument;
00018 using std::exception;
00019
00020 void menu() {
00021     vector<Studentas>grupe;
00022     int pas; // Pasirinkimas
00023     bool testi = true;
00024
00025     while (testi) {
00026         cout << string(80, '-') << "\n";
00027         cout << "Studentu Rezultatu skaiciavimo aplikacija\n";
00028         cout << string(80, '-') << "\n";
00029         cout << "1. Ivesti duomenis ranka \n";
00030         cout << "2. Generuoti tik pazymius \n";
00031         cout << "3. Generuoti studentu vardus, pavardes ir pazymius \n";
00032         cout << "4. Nuskaityti duomenis is failo \n";
00033         cout << "5. Sukurti testavimo failus (1000 - 10000000 irasu)\n";
00034         cout << "6. Atlikti spartos analize (nuskaitymas, rusiavimas, dalijimas, isvedimas)\n";
00035         cout << "7. Baigti darba \n";
00036
00037         try {
00038             if (!(cin >> pas)) {
00039                 throw invalid_argument("Neteisingas ivedimas! Iveskite skaiciu.");
00040             }
00041         }
00042         catch (const invalid_argument& e) {
00043             cout << "Klaida: " << e.what() << "\n";
00044             cin.clear();
00045             cin.ignore(1000, '\n');
00046             continue;
00047         }
00048
00049         // Pasirinkimo isvestis
00050         switch (pas) {
00051             case 1:
00052                 try {
00053                     cout << "1. Ivesti duomenisss ranka \n";
00054                     inputas(grupe);
00055                     outputas(grupe);
00056                     grupė.clear();
00057                 }
00058                 catch (const exception& e) {
00059                     cout << "Klaida vykstant 1 funkcija: " << e.what() << "\n";
00060                     grupė.clear();
00061                 }

```

```

00062         break;
00063     case 2:
00064         try {
00065             cout << "2. Generuoti tik pazymius \n";
00066             generuotiPaz(grupe);
00067             outputas(grupe);
00068             grupe.clear();
00069         }
00070         catch (const exception& e) {
00071             cout << "Klaida vykdant 2 funkcija: " << e.what() << "\n";
00072             grupe.clear();
00073         }
00074         break;
00075     case 3:
00076         try {
00077             cout << "3. Generuoti studentu vardus, pavardes ir pazymius \n";
00078             generuotiVardIrPav(grupe);
00079             outputas(grupe);
00080             grupe.clear();
00081         }
00082         catch (const exception& e) {
00083             cout << "Klaida vykdant 3 funkcija: " << e.what() << "\n";
00084             grupe.clear();
00085         }
00086         break;
00087     case 4:
00088         try {
00089             cout << "4. Nuskaityti duomenis is failo \n";
00090             skaitytiIsFailo(grupe);
00091             grupe.clear();
00092         }
00093         catch (const exception& e) {
00094             cout << "Klaida vykdant 4 funkcija: " << e.what() << "\n";
00095             grupe.clear();
00096         }
00097         break;
00098     case 5:
00099         try {
00100             cout << "5. Testavimo failu kurimas \n";
00101             sukurtiTestavimoFailus();
00102         }
00103         catch (const exception& e) {
00104             cout << "Klaida vykdant generavima: " << e.what() << "\n";
00105         }
00106         break;
00107     case 6:
00108         try {
00109             cout << "6. Spartos analizes vykdymas \n";
00110             atliktiSpartosAnalyze();
00111         }
00112         catch (const exception& e) {
00113             cout << "Klaida vykdant spartos analize: " << e.what() << "\n";
00114         }
00115         break;
00116     case 7:
00117         cout << "Programa uzdaroma \n";
00118         testi = false;
00119         break;
00120
00121     // Ivestis ivedus netinkama pasirinkima
00122     default:
00123         cout << "Klaida! Pasirinkite skaiciu nuo 1 iki 7 \n";
00124         cin.clear();
00125         cin.ignore(1000, '\n');
00126         break;
00127     }
00128 }
00129 }

```

5.5 output.cpp

```

00001 #include <iostream>
00002 #include <iomanip>
00003 #include <algorithm>
00004 #include <limits>
00005 #include <chrono>
00006
00007 #include "../header_files/output.h"
00008
00009 using std::cin;
00010 using std::cout;
00011 using std::string;
00012 using std::vector;

```

```

00013 using std::left;
00014 using std::right;
00015 using std::setw;
00016 using std::setprecision;
00017 using std::sort;
00018 using std::fixed;
00019 using std::numeric_limits;
00020 using std::streamsize;
00021 using std::chrono::high_resolution_clock;
00022 using std::chrono::duration;
00023
00024 void outputas(vector<Studentas>& grupe) {
00025     if (grupe.empty()) {
00026         cout << "Nera studentu duomenu!\n";
00027         return;
00028     }
00029
00030     // Klausiamas kaip rusiuoti
00031     cout << string(80, '-') << "\n";
00032     cout << "Pasirinkite rusiavimo buda:\n";
00033     cout << "1. Pagal varda (A-Z)\n";
00034     cout << "2. Pagal pavarde (A-Z)\n";
00035     cout << "3. Pagal galutini bala (vidurki) - didejimo tvarka\n";
00036     cout << "4. Pagal galutini bala (mediana) - didejimo tvarka\n";
00037
00038     int pasirinkimas;
00039     while (!(cin > pasirinkimas) || pasirinkimas < 1 || pasirinkimas > 4) {
00040         cout << "Klaida! Iveskite skaiciu nuo 1 iki 4: ";
00041         cin.clear();
00042         cin.ignore(numeric_limits<streamsize>::max(), '\n');
00043     }
00044
00045     cout << "Pasirinktas rusiavimas: " << pasirinkimas << "\n";
00046
00047     // Pradedamas rusiavimo laiko matavimas
00048     auto pradzia = high_resolution_clock::now();
00049
00050     // Rusiuojama pagal pasirinkima
00051     switch (pasirinkimas) {
00052     case 1:
00053         sort(grupe.begin(), grupe.end(), comparePagalVarda);
00054         break;
00055     case 2:
00056         sort(grupe.begin(), grupe.end(), comparePagalPavarde);
00057         break;
00058     case 3:
00059         sort(grupe.begin(), grupe.end(), comparePagalReza);
00060         break;
00061     case 4:
00062         sort(grupe.begin(), grupe.end(), comparePagalMeda);
00063         break;
00064     default:
00065         cout << "Klaida! Neteisingas pasirinkimas. Rodoma be rusiavimo.\n";
00066         break;
00067     }
00068
00069     // Baigiamas rusiavimo laiko matavimas
00070     auto pabaiga = high_resolution_clock::now();
00071     duration<double> trukme = pabaiga - pradzia;
00072
00073     // Antrastes eilute
00074     cout << string(80, '-') << "\n";
00075     cout << left << setw(20) << "Vardas" << setw(20) << "Pavarde"
00076         << setw(20) << "Galutinis (Vid.)" << setw(20) << "Galutinis (Med.)\n";
00077
00078     // Studentu duomenys
00079     for (const auto& A : grupe) {
00080         cout << left << setw(20) << A.getVardas() << setw(20) << A.getPavarde()
00081             << setw(20) << fixed << setprecision(2) << A.getRez()
00082             << setw(20) << fixed << setprecision(2) << A.getMed() << "\n";
00083     }
00084
00085     cout << string(80, '-') << "\n";
00086     cout << "Ivykdymo laikas: " << fixed << setprecision(7) << trukme.count() << " s\n";
00087 }

```

5.6 Studentas.cpp

```

00001 #include "../header_files/Studentas.h"
00002 #include "../header_files/mat_funkcijos.h"
00003 #include <iostream>
00004
00005 using std::istream;

```



```

00006
00007 // Nuskaitymas is srauto konstruktoriuje
00008 Studentas::Studentas(std::istream& is) : Zmogus(), egz_(0), rez_(0.0), med_(0.0) {
00009     readStudent(is);
00010 }
00011
00012 // ===== RULE OF FIVE =====
00013
00014 // Destruktorius
00015 Studentas::~Studentas() {
00016     paz_.clear();
00017 }
00018
00019 // Copy konstruktorius
00020 Studentas::Studentas(const Studentas& other)
00021 : Zmogus(other.vardas_, other.pavarde_, paz_(other.paz_),
00022     egz_(other.egz_), rez_(other.rez_), med_(other.med_)) {
00023 }
00024
00025 // Copy assignment operator
00026 Studentas& Studentas::operator=(const Studentas& other) {
00027     if (this != &other) {
00028         setVardas(other.getVardas());
00029         setPavarde(other.getPavarde());
00030         paz_ = other.paz_;
00031         egz_ = other.egz_;
00032         rez_ = other.rez_;
00033         med_ = other.med_;
00034     }
00035     return *this;
00036 }
00037
00038 // Move konstruktorius
00039 Studentas::Studentas(Studentas&& other) noexcept
00040 : Zmogus(std::move(other.vardas_), std::move(other.pavarde_)),
00041     paz_(std::move(other.paz_)), egz_(other.egz_), rez_(other.rez_),
00042     med_(other.med_) {
00043     other.setEgz(0);
00044     other.setRez(0.0);
00045     other.setMed(0.0);
00046     other.paz_.clear();
00047 }
00048
00049 // Move assignment operator
00050 Studentas& Studentas::operator=(Studentas&& other) noexcept {
00051     if (this != &other) {
00052         paz_.clear();
00053         setVardas(std::move(other.vardas_));
00054         setPavarde(std::move(other.pavarde_));
00055         paz_ = std::move(other.paz_);
00056         egz_ = other.egz_;
00057         rez_ = other.rez_;
00058         med_ = other.med_;
00059         other.setEgz(0);
00060         other.setRez(0.0);
00061         other.setMed(0.0);
00062         other.paz_.clear();
00063     }
00064     return *this;
00065 }
00066
00067 // Implementacija abstraktaus metodo iš Zmogus
00068 string Studentas::getInfo() const {
00069     return getVardas() + " " + getPavarde();
00070 }
00071
00072 // Privati pagalbine funkcija
00073 void Studentas::paskaiciuotiGalutinius() {
00074     if (!paz_.empty()) {
00075         double vid = vidurkis(paz_);
00076         double med = mediana(paz_);
00077         rez_ = galutinisBalas(vid, egz_);
00078         med_ = galutinisBalas(med, egz_);
00079     } else {
00080         rez_ = egz_ * 0.6;
00081         med_ = egz_ * 0.6;
00082     }
00083 }
00084
00085 // Implementacija abstraktaus metodo paskaiciuoti() iš Zmogus
00086 void Studentas::paskaiciuoti() {
00087     paskaiciuotiGalutinius();
00088 }
00089
00090 // Duomenų nuskaitymas ir apskaiciavimas
00091 std::istream& Studentas::readStudent(std::istream& is) {
00092     string vardas, pavarde;

```

```

00093     is » vardas » pavarde;
00094     if (!is) return is;
00095     setVardas(vardas);
00096     setPavarde(pavarde);
00097     paz_.clear();
00098     int p;
00099     for (int i = 0; i < 5; i++) {
00100         is » p;
00101         paz_.push_back(p);
00102     }
00103     is » egz_;
00104     paskaiciuotiGalutinius();
00105     return is;
00106 }
00107
00108 // ===== I/O OPERATORIAI =====
00109
00110 // Išvesties operatorius
00111 std::ostream& operator<<(std::ostream& os, const Studentas& s) {
00112     os << "Vardas: " << s.getVardas() << " | Pavarde: " << s.getPavarde() << " | ";
00113     os << "Egzaminas: " << s.egz_ << " | Vidurkis rezultatas: " << s.rez_ << " | ";
00114     os << "Mediana rezultatas: " << s.med_ << " | Pazymiai: ";
00115
00116     if (!s.paz_.empty()) {
00117         for (int paz : s.paz_) {
00118             os << paz << " ";
00119         }
00120     } else {
00121         os << "(nera)";
00122     }
00123
00124     return os;
00125 }
00126
00127 // ?vesties operatorius - nuskaityti studentą iš srauto
00128 std::istream& operator>>(std::istream& is, Studentas& s) {
00129     return s.readStudent(is);
00130 }
00131
00132 // ===== LYGINIMO FUNKCIJOS =====
00133
00134 bool comparePagalVarda(const Studentas& a, const Studentas& b) {
00135     return a.getVardas() < b.getVardas();
00136 }
00137
00138 bool comparePagalPavarde(const Studentas& a, const Studentas& b) {
00139     return a.getPavarde() < b.getPavarde();
00140 }
00141
00142 bool comparePagalReza(const Studentas& a, const Studentas& b) {
00143     return a.getRez() < b.getRez();
00144 }
00145
00146 bool comparePagalMeda(const Studentas& a, const Studentas& b) {
00147     return a.getMed() < b.getMed();
00148 }

```

5.7 testavimas.cpp

```

00001 #include <iostream>
00002 #include <fstream>
00003 #include <vector>
00004 #include <string>
00005 #include <chrono>
00006 #include <iomanip>
00007 #include <random>
00008 #include <algorithm>
00009 #include <stdexcept>
00010 #include <list>
00011 #include <deque>
00012 #include <type_traits>
00013 #include <iterator> // std::make_move_iterator
00014
00015 #include "../header_files/testavimas.h"
00016 #include "../header_files/mat_funkcijos.h"
00017
00018 using std::cout;
00019 using std::cin;
00020 using std::vector;
00021 using std::string;
00022 using std::ofstream;
00023 using std::ifstream;
00024 using std::chrono::high_resolution_clock;

```

```

00025 using std::chrono::duration;
00026 using std::fixed;
00027 using std::setprecision;
00028 using std::left;
00029 using std::setw;
00030 using std::to_string;
00031 using std::move;
00032 using std::list;
00033 using std::deque;
00034
00035 const vector<int> SZ = { 1000, 10000, 100000, 1000000, 10000000 };
00036 const int paz_kiekis = 5;
00037
00038 void sukurtiTestavimoFailus() {
00039     int pas;
00040     cout << "\nPasirinkite, kuri faila norite sukurti:\n";
00041     cout << "1. 1000 irasu\n";
00042     cout << "2. 10000 irasu\n";
00043     cout << "3. 100000 irasu\n";
00044     cout << "4. 1 000000 irasu\n";
00045     cout << "5. 10000000 irasu\n";
00046     cout << "6. Visus auksciau isvardintus\n";
00047     cout << "Jusu pasirinkimas: ";
00048     cin >> pas;
00049
00050     vector<int> pasirinkti_SZ;
00051     if (pas == 1) pasirinkti_SZ = { 1000 };
00052     else if (pas == 2) pasirinkti_SZ = { 10000 };
00053     else if (pas == 3) pasirinkti_SZ = { 100000 };
00054     else if (pas == 4) pasirinkti_SZ = { 1000000 };
00055     else if (pas == 5) pasirinkti_SZ = { 10000000 };
00056     else if (pas == 6) pasirinkti_SZ = SZ;
00057     else {
00058         cout << "Neteisingas pasirinkimas!\n";
00059         return;
00060     }
00061
00062     srand(static_cast<unsigned>(time(nullptr)));
00063     cout << "Failu kurimas prasidejo...\n";
00064     cout << string(80, '-') << "\n";
00065
00066     for (int n : pasirinkti_SZ) {
00067         string fname = "Studentai_test\\studentai_" + to_string(n) + ".txt";
00068
00069         // Pradedamas matuoti laikas tiems irasams kurti
00070         auto start = high_resolution_clock::now();
00071
00072         ofstream out(fname);
00073
00074         // Antraste
00075         out << left << setw(20) << "Vardas" << setw(20) << "Pavarde";
00076         for (int i = 1; i <= paz_kiekis; i++) {
00077             out << setw(8) << ("ND" + to_string(i));
00078         }
00079         out << "Egz\n";
00080
00081         // Tiesiai irasome i faila (be vektorių pagal uzduoties salyga)
00082         for (int i = 1; i <= n; i++) {
00083             out << left << setw(20) << ("Vardas" + to_string(i)) << setw(20) << ("Pavarde" + to_string(i));
00084             for (int k = 0; k < paz_kiekis; k++) {
00085                 out << setw(8) << (rand() % 10 + 1);
00086             }
00087             out << (rand() % 10 + 1) << "\n";
00088         }
00089         out.close();
00090
00091         // Stabdyti laikmati ir isvesti i ekrana laika
00092         auto end = high_resolution_clock::now();
00093         duration<double> diff = end - start;
00094
00095         cout << n << " irasu failo sukurimo laikas: " << fixed << setprecision(5) << diff.count() << "
00096             s.\n";
00097     }
00098 }
00099
00099 template <typename Container>
00100 void nuskaitytiDuomenis(const string& fname, Container& grupe) {
00101     ifstream in(fname);
00102     string header;
00103     std::getline(in, header);
00104
00105     Studentas st;
00106     st.reservePaz(paz_kiekis);
00107
00108     while (true) {
00109         string v, p;
00110         if (!(in >> v >> p)) break;

```

```

00111         st.setVardas(v);
00112         st.setPavarde(p);
00113         st.clearPaz();
00114         int pr;
00115         for (int i = 0; i < paz_kiekis; i++) {
00116             in » pr;
00117             st.addPaz(pr);
00118         }
00119         int egz; in » egz; st.setEgz(egz);
00120         st.setRez(galutinisBalas(vidurkis(st.getPaz()), st.getEgz()));
00121         grupe.push_back(st);
00122     }
00123     in.close();
00124 }
00125
00126 template <typename Container>
00127 double tirtiKonteineri(int n, const string& fname, int strat) {
00128     auto total_start = high_resolution_clock::now();
00129
00130     // 1. Duomenų nuskaitymas iš failo
00131     auto start = high_resolution_clock::now();
00132
00133     Container grupe;
00134     if constexpr (std::is_same_v<Container, vector<Studentas>) {
00135         try {
00136             grupe.reserve(n); // list neturi reserve() funkcijos
00137         } catch (const std::exception& e) {
00138             cout << "Nepakanka atminties rezervuoti " << n << " elementu!\n";
00139             return 0.0;
00140         }
00141     }
00142
00143     nuskaitytiDuomenis(fname, grupe);
00144
00145     auto end = high_resolution_clock::now();
00146     duration<double> ms_read = end - start;
00147     cout << n << " Irasu failo nuskaitymo laikas: " << fixed << setprecision(5) << ms_read.count() << "
00148         s.\n";
00149
00150     // 2. Rūšiavimas
00151     start = high_resolution_clock::now();
00152     if constexpr (std::is_same_v<Container, list<Studentas>) {
00153         grupe.sort([](const Studentas& a, const Studentas& b) {
00154             return a.getRez() < b.getRez();
00155         });
00156     } else {
00157         std::sort(grupe.begin(), grupe.end(), [](const Studentas& a, const Studentas& b) {
00158             return a.getRez() < b.getRez();
00159         });
00160     }
00161     end = high_resolution_clock::now();
00162     duration<double> ms_sort = end - start;
00163     cout << n << " Irasu rūšiavimas didejimo tvarka: " << fixed << setprecision(5) << ms_sort.count() << "
00164         s.\n";
00165
00166     // 3. Skirstymas į dvi grupes
00167     start = high_resolution_clock::now();
00168     Container vargiukai;
00169     Container kietiakai;
00170
00171     if (strat == 1) {
00172         if constexpr (std::is_same_v<Container, vector<Studentas>) {
00173             vargiukai.reserve(n / 2 + 100);
00174             kietiakai.reserve(n / 2 + 100);
00175         }
00176         for (auto& s : grupe) {
00177             if (s.getRez() < 5.0) {
00178                 vargiukai.push_back(move(s));
00179             }
00180             else {
00181                 kietiakai.push_back(move(s));
00182             }
00183         }
00184         Container().swap(grupe); // Išvalome pradinius konteinerius
00185     } else if (strat == 2) {
00186         if constexpr (std::is_same_v<Container, vector<Studentas>) {
00187             // Sukrupuoja vektorius vietoje (O(N) laikas):
00188             auto splitPoint = std::partition(grupe.begin(), grupe.end(), [](const Studentas& s) {
00189                 return s.getRez() >= 5.0; // Salyga kietiakams
00190             });
00191
00192             // I anksto rezervuojame atminties vargiukams (nebutina, bet dar labiau pagreitina)
00193             vargiukai.reserve(std::distance(splitPoint, grupe.end()));
00194
00195             // NAUDOJAME MOVE: Perkeliame vargiukus į pagrindinio vektoriaus galo į naujais.

```

```

00196         vargsiukai.insert(vargsiukai.end(),
00197                             std::make_move_iterator(splitPoint),
00198                             std::make_move_iterator(grupe.end()));
00199
00200         // Itriname perkeltus elementus i originalaus vektoriaus.
00201         grupe.erase(splitPoint, grupe.end());
00202         kietiakai = move(grupe); // Like grupeje yra kietiakai
00203     }
00204     else {
00205         auto it = grupe.begin();
00206         while (it != grupe.end()) {
00207             if (it->getRez() < 5.0) {
00208                 vargsiukai.push_back(move(*it));
00209                 it = grupe.erase(it); // vector/deque atveju tai leta, list atveju - greita
00210             }
00211             else {
00212                 ++it;
00213             }
00214         }
00215         kietiakai = move(grupe); // Like grupeje yra kietiakai
00216     }
00217 }
00218 else if (strat == 3) {
00219     if constexpr (std::is_same_v<Container, list<Studentas>) {
00220         // list atveju efektyviausia yra ikirpti elementus per splice + remove_if ar pagal salyga
00221         auto it = std::partition(grupe.begin(), grupe.end(), [](const Studentas& s) {
00222             return s.getRez() < 5.0; // Vargiukaikeliauja i pati prieki
00223         });
00224         // Ikerpame vargiukus i grupes tiesiai i vargiuku list be memory re-allocation
00225         vargsiukai.splice(vargsiukai.begin(), grupe, grupe.begin(), it);
00226         kietiakai = move(grupe);
00227     }
00228     else {
00229         // std::vector ir std::deque atveju efektyviausia naudoti std::partition in-place
00230         if constexpr (std::is_same_v<Container, vector<Studentas>) {
00231             vargsiukai.reserve(n / 2 + 100);
00232         }
00233         auto it = std::stable_partition(grupe.begin(), grupe.end(), [](const Studentas& s) {
00234             return s.getRez() < 5.0;
00235         });
00236         // Iteruojam per pradia kur atsidure vargiukai po stable_partition pakeitimu
00237         vargsiukai.insert(vargsiukai.end(), std::make_move_iterator(grupe.begin()),
00238                         std::make_move_iterator(it));
00239         // Kas liko originalioje grupėje iteruojame i kietiakių (nuo it iki end)
00240         kietiakai.insert(kietiakai.end(), std::make_move_iterator(it),
00241                         std::make_move_iterator(grupe.end()));
00242     }
00243     Container().swap(grupe); // Sunaikinam originala
00244 }
00245
00246 end = high_resolution_clock::now();
00247 duration<double> ms_split = end - start;
00248 cout << n << " Irasu dalijimo laikas (" << strat << "-a strategija): " << fixed << setprecision(5) <<
00249 ms_split.count() << " s.\n";
00250
00251 // 4.1 Irasome vargiukus i faila
00252 start = high_resolution_clock::now();
00253 ofstream outV("vargsiukai\\vargsiukai_" + to_string(n) + ".txt");
00254 outV << left << setw(20) << "Vardas" << setw(20) << "Pavarde" << "Galutinis (Vid.)\n";
00255 for (const auto& s : vargsiukai) {
00256     outV << left << setw(20) << s.getVardas() << setw(20) << s.getPavarde() << fixed << setprecision(2) <<
00257     s.getRez() << "\n";
00258 }
00259 outV.close();
00260 Container().swap(vargsiukai); // Islaisviname RAM perrasydami!
00261 end = high_resolution_clock::now();
00262 duration<double> ms_write_v = end - start;
00263 cout << n << " Irasu 'vargsiuku' irasyimo laikas: " << fixed << setprecision(5) << ms_write_v.count() <<
00264 " s.\n";
00265
00266 // 4.2 Irasome kietiakių i faila
00267 start = high_resolution_clock::now();
00268 ofstream outK("kietiakai\\kietiakai_" + to_string(n) + ".txt");
00269 outK << left << setw(20) << "Vardas" << setw(20) << "Pavarde" << "Galutinis (Vid.)\n";
00270 for (const auto& s : kietiakai) {
00271     outK << left << setw(20) << s.getVardas() << setw(20) << s.getPavarde() << fixed << setprecision(2) <<
00272     s.getRez() << "\n";
00273 }
00274 outK.close();
00275 Container().swap(kietiakai); // Islaisviname ir kietiakių!
00276 end = high_resolution_clock::now();
00277 duration<double> ms_write_k = end - start;
00278 cout << n << " Irasu 'kietiaku' irasyimo laikas: " << fixed << setprecision(5) << ms_write_k.count() <<
00279 " s.\n";
00280

```

```

00276 // Bendras laikas
00277 auto total_end = high_resolution_clock::now();
00278 duration<double> ms_total = total_end - total_start;
00279 return ms_total.count();
00280 }
00281
00282 void atliktiSpartosAnalyze() {
00283     int pas, kont, strat, test_kartai;
00284     cout << "\nPasirinkite, su kuriuo failu norite atlikti spartos analize:\n";
00285     cout << "1. 1000 irasu failas\n";
00286     cout << "2. 10000 irasu failas\n";
00287     cout << "3. 100000 irasu failas\n";
00288     cout << "4. 1000000 irasu failas\n";
00289     cout << "5. 10000000 irasu failas\n";
00290     cout << "6. Visi failai is eiles\n";
00291     cout << "Jusu pasirinkimas: ";
00292     cin >> pas;
00293
00294     vector<int> pasirinkti_SZ;
00295     if (pas == 1) pasirinkti_SZ = { 1000 };
00296     else if (pas == 2) pasirinkti_SZ = { 10000 };
00297     else if (pas == 3) pasirinkti_SZ = { 100000 };
00298     else if (pas == 4) pasirinkti_SZ = { 1000000 };
00299     else if (pas == 5) pasirinkti_SZ = { 10000000 };
00300     else if (pas == 6) pasirinkti_SZ = SZ;
00301     else {
00302         cout << "Neteisingas pasirinkimas!\n";
00303         return;
00304     }
00305
00306     cout << "\nPasirinkite konteinerio tipa testavimui:\n";
00307     cout << "1. std::vector\n";
00308     cout << "2. std::list\n";
00309     cout << "3. std::deque\n";
00310     cout << "Jusu pasirinkimas: ";
00311     cin >> kont;
00312
00313     cout << "\nPasirinkite testavimo strategija:\n";
00314     cout << "1. 1 strategija (Sukurti 2 naujus konteinerius)\n";
00315     cout << "2. 2 strategija (Sukurti 1 nauja konteineri, trinti is pagrindinio)\n";
00316     cout << "3. 3 strategija (Optimizuoti algoritmu metodai)\n";
00317     cout << "Jusu pasirinkimas: ";
00318     cin >> strat;
00319
00320     cout << "\nIveskite, kiek kartu norite pakartoti testavima (laiku vidurkiui): ";
00321     cin >> test_kartai;
00322
00323     cout << "\nPradedama spartos analize pagal nurodytus zingsnius...\n";
00324
00325     for (int n : pasirinkti_SZ) {
00326         string fname = "Studentai_test\\studentai_" + to_string(n) + ".txt";
00327
00328         // Tikrinimas ar failas atsidaro nenaudojant filesystem
00329         ifstream patikrinimas(fname);
00330         if (!patikrinimas.good()) {
00331             cout << "Failo " << fname << " nera. Praleidžiama.\n";
00332             continue;
00333         }
00334         patikrinimas.close();
00335
00336         cout << string(80, '-') << "\n";
00337         cout << n << " Irasu spartos analize\n";
00338         cout << string(80, '-') << "\n";
00339
00340         double visas_laikas = 0.0;
00341
00342         for (int i = 0; i < test_kartai; i++) {
00343             cout << "nTESTO NUMERIS: " << i + 1 << "\n";
00344             if (kont == 1) visas_laikas += tirtiKonteineri<vector<Studentas>(n, fname, strat);
00345             else if (kont == 2) visas_laikas += tirtiKonteineri<list<Studentas>(n, fname, strat);
00346             else if (kont == 3) visas_laikas += tirtiKonteineri<deque<Studentas>(n, fname, strat);
00347             else {
00348                 cout << "Neteisingas konteinerio tipas!\n";
00349                 return;
00350             }
00351         }
00352         cout << "\n===== \n";
00353         cout << "Vidutinis viso testo (nuskaitymas+dalijimas+isvedimas) laikas po " << test_kartai << "
00354         bandymu:\n";
00355         cout << fixed << setprecision(5) << visas_laikas / test_kartai << " s.\n";
00356         cout << "===== \n\n";
00357     }
00358
00359 void PadalintiStudentusZabioGreiciu(std::vector<Studentas>& studentai, std::vector<Studentas>&
00360     vargsiukai) {

```

```

00361 // 1. Sukrupuojame vektoriu vietoje (O(N) laikas):
00362 // Visi, kuriu balas >= 5.0 atsiduria priekyje, o < 5.0 - gale.
00363 // Jei jums BUTINA ilaikyti jau esama eilikuma (pvz., abecelini),
00364 // vietoj 'std::partition' naudokite 'std::stable_partition'.
00365 auto splitPoint = std::partition(studentai.begin(), studentai.end(), [](const Studentas& s) {
00366     return s.getRez() >= 5.0; // Pakeista is galutinis i rez
00367 });
00368
00369 // 2. I anksto rezervuojame atminties vargiukams (nebutina, bet dar labiau pagreitina)
00370 vargsiukai.reserve(std::distance(splitPoint, studentai.end()));
00371
00372 // 3. NAUDOJAME MOVE: Perkeliame vargiukus i pagrindinio vektoriaus galo i naujaji.
00373 // std::make_move_iterator utikrina, kad tekstai ir kiti duomenys nebutu kopijuojami i naujo.
00374 vargsiukai.insert(vargsiukai.end(),
00375                 std::make_move_iterator(splitPoint),
00376                 std::make_move_iterator(studentai.end()));
00377
00378 // 4. Itriname perkeltus elementus i originalaus vektoriaus.
00379 // Kadangi triname I GALO, jokio duomeniu stumdymo nebelieka - operacija ivyksta per O(1).
00380 studentai.erase(splitPoint, studentai.end());
00381 }

```

5.8 Zmogus.cpp

```

00001 #include "../header_files/Zmogus.h"
00002 #include <iostream>
00003
00004 using std::ostream;
00005
00006 /**
00007 * @brief Išvesties operatorius - spausdina žmogaus pagrindinius duomenis
00008 * @param os Išvesties srautas
00009 * @param z Žmogaus objektas
00010 * @return Išvesties srautas
00011 */
00012 std::ostream& operator<<(std::ostream& os, const Zmogus& z) {
00013     os << "Vardas: " << z.vardas_ << " | Pavarde: " << z.pavarde_;
00014     return os;
00015 }

```

5.9 duomeniu_valdymas.h

```

00001 #pragma once
00002
00003 #include <vector>
00004 #include <string>
00005
00006 #include "studentas.h"
00007
00008 using std::string;
00009 using std::vector;
00010
00011 void generuotiPaz(vector<Studentas>& grupe);
00012 void generuotiVardIrPav(vector<Studentas>& grupe);
00013 void skaitytiIsFailo(vector<Studentas>& grupe);
00014 void rasytiIFaila(vector<Studentas>& grupe);

```

5.10 input.h

```

00001 #pragma once
00002
00003 #include <vector>
00004 #include <string>
00005 #include "studentas.h"
00006
00007 using std::vector;
00008 using std::string;
00009
00010 void inputas(vector<Studentas>& grupe);

```

5.11 Mat_funkcijos.h

```
00001 #pragma once
00002
00003 #include <string>
00004 #include <vector>
00005
00006 using std::string;
00007 using std::vector;
00008
00009 bool arTikRaides(const string& str);
00010 double mediana(vector<int> paz);
00011 double vidurkis(const vector<int>& paz); // Priima per referencą greičiui
00012 double galutinisBalas(double vidMed, int egz);
00013 int generuotiPazymi();
00014 int generuotiEgzamina();
00015
```

5.12 menu.h

```
00001 #pragma once
00002
00003 void menu();
00004
```

5.13 output.h

```
00001 #pragma once
00002
00003 #include <vector>
00004 #include <string>
00005
00006 #include "studentas.h"
00007
00008 using std::vector;
00009 using std::string;
00010
00011 void outputas(vector<Studentas>& grupe);
```

5.14 Studentas.h

```
00001 #pragma once
00002
00003 #include <iostream>
00004 #include <string>
00005 #include <vector>
00006 #include "Zmogus.h"
00007
00008 using std::string;
00009 using std::vector;
00010
00011 // Išvestinė klasė iš Zmogus - aprašo studentą
00012 // Paveldi žmogaus duomenis (vardas, pavardė) iš Zmogus klasės
00013 // Turi papildomus duomenis: pažymius, egzamino rezultatą ir galutinius rezultatus
00014 // Implementuoja Rule of Five
00015 class Studentas : public Zmogus {
00016 private:
00017     vector<int> paz_;
00018     int egz_;
00019     double rez_;
00020     double med_;
00021
00022     // Privatas helperis skaičiavimams
00023     void paskaiciuotiGalutinius();
00024
00025     // Implementacija abstraktaus metodo iš Zmogus
00026     virtual void paskaiciuoti() override;
00027
00028 public:
00029     // Numatytasis konstruktorius
00030     Studentas() : Zmogus("A", "BB"), egz_(0), rez_(0.0), med_(0.0) {}
00031
00032     // Parametrizuotas konstruktorius
```



```

00033     Studentas(string v, string p, int e) : Zmogus(v, p), egz_(e), rez_(0.0), med_(0.0) {}
00034
00035     // Konstruktorius su nuskaitymu is srauto
00036     Studentas(std::istream& is);
00037
00038     // Rule of Five
00039
00040     // Destruktorius
00041     ~Studentas();
00042
00043     // Copy konstruktorius
00044     Studentas(const Studentas& other);
00045
00046     // Copy assignment operator
00047     Studentas& operator=(const Studentas& other);
00048
00049     // Move konstruktorius
00050     Studentas(Studentas&& other) noexcept;
00051
00052     // Move assignment operator
00053     Studentas& operator=(Studentas&& other) noexcept;
00054
00055     // Getters
00056     inline const vector<int>& getPaz() const { return paz_; }
00057     inline vector<int>& getPaz() { return paz_; }
00058     inline int getEgz() const { return egz_; }
00059     inline double getRez() const { return rez_; }
00060     inline double getMed() const { return med_; }
00061
00062     // Setters
00063     inline void setEgz(int e) { egz_ = e; }
00064     inline void setRez(double r) { rez_ = r; }
00065     inline void setMed(double m) { med_ = m; }
00066
00067     // Darbas su pažymiais
00068     inline void addPaz(int p) { paz_.push_back(p); }
00069     inline void clearPaz() { paz_.clear(); }
00070     inline void reservePaz(size_t n) { paz_.reserve(n); }
00071     inline bool isPazEmpty() const { return paz_.empty(); }
00072     inline void setPaz(const vector<int>& p) { paz_ = p; }
00073
00074     // Skaitymas ir skaičiavimas
00075     std::istream& readStudent(std::istream& is);
00076
00077     // Implementacija abstraktaus metodo iš Zmogus
00078     virtual string getInfo() const override;
00079
00080     // I/O operatoriai
00081     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00082     friend std::istream& operator>>(std::istream& is, Studentas& s);
00083 };
00084
00085 // Ne klasės narės, bet su klase tiesiogiai susijusios lyginimo funkcijos
00086 bool comparePagalVarda(const Studentas& a, const Studentas& b);
00087 bool comparePagalPavarde(const Studentas& a, const Studentas& b);
00088 bool comparePagalReza(const Studentas& a, const Studentas& b);
00089 bool comparePagalMeda(const Studentas& a, const Studentas& b);
00090

```

5.15 testavimas.h

```

00001 #pragma once
00002 #include <vector>
00003 #include "studentas.h"
00004
00005 void sukurtiTestavimoFailus();
00006 void atliktiSpartosAnalyze();

```

5.16 Zmogus.h

```

00001 #pragma once
00002
00003 #include <iostream>
00004 #include <string>
00005
00006 using std::string;
00007
00008 /**

```

```

00009  * @class Zmogus
00010  * @brief Abstrakti bazinė klasė bendram žmogaus aprašymui
00011  *
00012  * Ši klasė nuo v1.5 versijos aprašo bendrą žmogaus duomenis (vardas, pavardė).
00013  * Yra abstrakti - jos objektų kurti negalima, tik iš jos išvestinės klasės.
00014  *
00015  * Paveldi iš šios klasės: Studentas
00016  */
00017 class Zmogus {
00018 protected:
00019     string vardas_;
00020     string pavarde_;
00021
00022     /**
00023      * @brief Pagalbinis metodas žmogaus duomenims paskaičiuoti
00024      * Abstraktus metodas - turi būti implementuotas išvestinėse klasėse
00025      */
00026     virtual void paskaiciuoti() = 0;
00027
00028 public:
00029     /**
00030      * @brief Parametrizuotas konstruktorius
00031      * @param v Žmogaus vardas
00032      * @param p Žmogaus pavardė
00033      */
00034     Zmogus(const string& v, const string& p)
00035         : vardas_(v), pavarde_(p) {}
00036
00037     /**
00038      * @brief Parametrizuotas konstruktorius su numatytosiomis reikšmėmis
00039      */
00040     Zmogus() : vardas_("Asmuo"), pavarde_("Asmenys") {}
00041
00042     /**
00043      * @brief Virtualus destruktorius - reikalingas polimorfizmui
00044      */
00045     virtual ~Zmogus() {}
00046
00047     // ===== GETTERS =====
00048     /**
00049      * @return Žmogaus vardą
00050      */
00051     inline string getVardas() const { return vardas_; }
00052
00053     /**
00054      * @return Žmogaus pavardę
00055      */
00056     inline string getPavarde() const { return pavarde_; }
00057
00058     // ===== SETTERS =====
00059     /**
00060      * @brief Nustato žmogaus vardą
00061      */
00062     inline void setVardas(const string& v) { vardas_ = v; }
00063
00064     /**
00065      * @brief Nustato žmogaus pavardę
00066      */
00067     inline void setPavarde(const string& p) { pavarde_ = p; }
00068
00069     // ===== ABSTRAKTUS METODAS =====
00070     /**
00071      * @brief Gražina žmogaus informaciją
00072      * Šis metodas turi būti implementuotas visose išvestinėse klasėse
00073      */
00074     virtual string getInfo() const = 0;
00075
00076     // ===== I/O OPERATORIAI (friend functions) =====
00077     /**
00078      * @brief Išvesties operatorius - spausdina žmogaus duomenis
00079      */
00080     friend std::ostream& operator<<(std::ostream& os, const Zmogus& z);
00081 };

```

5.17 objekt1uzd.cpp

```

00001 #include "header_files/menu.h"
00002
00003 int main() {
00004     menu();
00005     return 0;
00006 }

```

5.18 test_studentas.cpp

```

00001 #include <gtest/gtest.h>
00002 #include <iostream>
00003 #include <sstream>
00004 #include <cassert>
00005 #include <vector>
00006 #include "header_files/Studentas.h"
00007
00008 using std::cout;
00009 using std::cin;
00010 using std::endl;
00011 using std::string;
00012 using std::vector;
00013 using std::stringstream;
00014
00015 int testai_praletii = 0;
00016 int testai_nepraletii = 0;
00017
00018 void printTestHeader(const string& testName) {
00019     cout << "\n" << string(60, '=') << endl;
00020     cout << "TEST: " << testName << endl;
00021     cout << string(60, '=') << endl;
00022 }
00023
00024 void testPassed(const string& info) {
00025     cout << "[PRIIMTAS] " << info << endl;
00026     testai_praletii++;
00027 }
00028
00029 void testFailed(const string& info) {
00030     cout << "[NEPRIIMTAS] " << info << endl;
00031     testai_nepraletii++;
00032 }
00033
00034 // TEST #1: Default konstruktorius
00035 void test_default_constructor() {
00036     printTestHeader("Default Konstruktorius");
00037
00038     Studentas s;
00039
00040     assert(s.getVardas() == "A");
00041     testPassed("Vardas inicijuotas i 'A'");
00042
00043     assert(s.getPavarde() == "BB");
00044     testPassed("Pavarde inicijuota i 'BB'");
00045
00046     assert(s.getEgz() == 0);
00047     testPassed("Egzaminas inicijuotas i 0");
00048
00049     assert(s.getRez() == 0.0);
00050     testPassed("Rezultatas inicijuotas i 0.0");
00051
00052     assert(s.getMed() == 0.0);
00053     testPassed("Mediana inicijuota i 0.0");
00054
00055     assert(s.isPazEmpty());
00056     testPassed("Pazymiai sarasas tucias");
00057 }
00058
00059 // TEST #2: Parametrizuotas konstruktorius
00060 void test_parametrized_constructor() {
00061     printTestHeader("Parametrizuotas Konstruktorius");
00062
00063     Studentas s("Jonas", "Jonaitis", 95);
00064
00065     assert(s.getVardas() == "Jonas");
00066     testPassed("Vardas teisingai priskirtas");
00067
00068     assert(s.getPavarde() == "Jonaitis");
00069     testPassed("Pavarde teisingai priskirta");
00070
00071     assert(s.getEgz() == 95);
00072     testPassed("Egzaminas teisingai priskirtas");
00073 }
00074
00075 // TEST #3: Copy konstruktorius
00076 void test_copy_constructor() {
00077     printTestHeader("Copy Konstruktorius");
00078
00079     Studentas s1("Petras", "Petrauskas", 85);
00080     s1.addPaz(8);
00081     s1.addPaz(9);
00082     s1.addPaz(10);
00083
00084     // Kuriam kopija

```

```
00085     Studentas s2 = s1;
00086
00087     assert(s2.getVardas() == s1.getVardas());
00088     testPassed("Kopijoje vardas sutampa");
00089
00090     assert(s2.getPavarde() == s1.getPavarde());
00091     testPassed("Kopijoje pavarde sutampa");
00092
00093     assert(s2.getEgz() == s1.getEgz());
00094     testPassed("Kopijoje egzaminas sutampa");
00095
00096     assert(s2.getPaz().size() == s1.getPaz().size());
00097     testPassed("Kopijoje pazymiu kiekis sutampa");
00098
00099     assert(s2.getPaz()[0] == s1.getPaz()[0]);
00100     testPassed("Kopijoje pazymiu vertes sutampa");
00101
00102     // Tikriname, kad tai skirtingi objektai (deep copy)
00103     Studentas s3 = s1;
00104     s3.addPaz(100);
00105     assert(s1.getPaz().size() != s3.getPaz().size());
00106     testPassed("Deep copy veikia - modifikacija nepaveikia originalo");
00107 }
00108
00109 // TEST #4: Copy assignment operator
00110 void test_copy_assignment() {
00111     printTestHeader("Copy Assignment Operatorius");
00112
00113     Studentas s1("Marta", "Martine", 92);
00114     s1.addPaz(7);
00115     s1.addPaz(8);
00116     s1.addPaz(9);
00117
00118     Studentas s2;
00119     s2 = s1;
00120
00121     assert(s2.getVardas() == "Marta");
00122     testPassed("Priskyrimo metu vardas is naujo priskirtas");
00123
00124     assert(s2.getPavarde() == "Martine");
00125     testPassed("Priskyrimo metu pavarde is naujo priskirta");
00126
00127     assert(s2.getPaz().size() == 3);
00128     testPassed("Priskyrimo metu pazymiai is naujo priskirti");
00129
00130     // Self-assignment check
00131     s2 = s2;
00132     assert(s2.getVardas() == "Marta");
00133     testPassed("Self-assignment saugiai veikia");
00134 }
00135
00136 // TEST #5: Move konstruktorius
00137 void test_move_constructor() {
00138     printTestHeader("Move Konstruktorius");
00139
00140     Studentas s1("Laima", "Laima", 88);
00141     s1.addPaz(6);
00142     s1.addPaz(7);
00143     s1.addPaz(8);
00144
00145     // Move konstruktorius
00146     Studentas s2 = std::move(s1);
00147
00148     assert(s2.getVardas() == "Laima");
00149     testPassed("Move konstruktorius - vardas is naujo priskirtas");
00150
00151     assert(s2.getPaz().size() == 3);
00152     testPassed("Move konstruktorius - pazymiai is naujo priskirti");
00153
00154     // s1 turetu tureti tuscius pazymius
00155     assert(s1.isPazEmpty());
00156     testPassed("Move konstruktorius - saltinis istustintas");
00157 }
00158
00159 // TEST #6: Move assignment operatorius
00160 void test_move_assignment() {
00161     printTestHeader("Move Assignment Operatorius");
00162
00163     Studentas s1("Ruta", "Rute", 90);
00164     s1.addPaz(5);
00165     s1.addPaz(6);
00166     s1.addPaz(7);
00167
00168     Studentas s2;
00169     s2 = std::move(s1);
00170
00171     assert(s2.getVardas() == "Ruta");
```

```

00172     testPassed("Move assignment - vardas is naujo priskirtas");
00173
00174     assert(s2.getPaz().size() == 3);
00175     testPassed("Move assignment - pazymiai is naujo priskirti");
00176
00177     assert(s1.isPazEmpty());
00178     testPassed("Move assignment - saltinis istustintas");
00179
00180     // Self-move assignment check
00181     s2 = std::move(s2);
00182     assert(s2.getVardas() == "Ruta");
00183     testPassed("Move assignment self-assignment saugiai veikia");
00184 }
00185
00186 // TEST #7: Destruktorius - RAII principas
00187 void test_destructor() {
00188     printTestHeader("Destruktorius - RAII Principas");
00189
00190     {
00191         Studentas s("Vidmantas", "Vidmante", 87);
00192         s.addPaz(9);
00193         s.addPaz(10);
00194         s.addPaz(8);
00195
00196         assert(s.getPaz().size() == 3);
00197         testPassed("Destruktoriaus testas - objektas sukurtas");
00198     } // Cia is automatiskai kvieciamas destruktoriaus
00199
00200     testPassed("Destruktor veikia be problemu (nera memory leaks)");
00201 }
00202
00203 // TEST #8: Isvesties operatorius (operator<>)
00204 void test_output_operator() {
00205     printTestHeader("Isvesties Operatorius (operator<>)");
00206
00207     Studentas s("Darius", "Darius", 93);
00208     s.addPaz(9);
00209     s.addPaz(10);
00210     s.addPaz(9);
00211
00212     stringstream ss;
00213     ss << s;
00214     string output = ss.str();
00215
00216     assert(output.find("Vardas: Darius") != string::npos);
00217     testPassed("Isvestis - vardas rastas");
00218
00219     assert(output.find("Pavarde: Darius") != string::npos);
00220     testPassed("Isvestis - pavarde rastas");
00221
00222     assert(output.find("Egzaminas: 93") != string::npos);
00223     testPassed("Isvestis - egzaminas rastas");
00224
00225     cout << "Isvesties pavyzdys: " << endl;
00226     cout << " " << s << endl;
00227 }
00228
00229 // TEST #9: Ivesties operatorius (operator>>)
00230 void test_input_operator() {
00231     printTestHeader("Ivesties Operatorius (operator>>)");
00232
00233     stringstream ss;
00234     ss << "Vaidas Vaidauskas 10 10 10 10 10 100";
00235
00236     Studentas s;
00237     ss >> s;
00238
00239     assert(s.getVardas() == "Vaidas");
00240     testPassed("Ivestis - vardas nuskaitytas");
00241
00242     assert(s.getPavarde() == "Vaidauskas");
00243     testPassed("Ivestis - pavarde nuskaityta");
00244
00245     assert(s.getEgz() == 100);
00246     testPassed("Ivestis - egzaminas nuskaitytas");
00247
00248     assert(s.getPaz().size() == 5);
00249     testPassed("Ivestis - pazymiai nuskaityti");
00250
00251     assert(s.getPaz()[0] == 10);
00252     testPassed("Ivestis - pazymio reiksme teisingai nuskaityta");
00253 }
00254
00255 // TEST #10: I/O operatoriai kartu (Round-trip test)
00256 void test_io_roundtrip() {
00257     printTestHeader("I/O Round-Trip Testas");
00258

```

```

00259     Studentas s1("Kristina", "Kristina", 96);
00260     s1.addPaz(8);
00261     s1.addPaz(9);
00262     s1.addPaz(10);
00263     s1.addPaz(9);
00264     s1.addPaz(8);
00265
00266     stringstream ss;
00267     ss << s1.getVardas() << " " << s1.getPavarde();
00268     for (int paz : s1.getPaz()) {
00269         ss << " " << paz;
00270     }
00271     ss << " " << s1.getEgz();
00272
00273     Studentas s2;
00274     ss >> s2;
00275
00276     assert(s2.getVardas() == s1.getVardas());
00277     testPassed("Round-trip - vardas saugai issaugotas ir nuskaitytas");
00278
00279     assert(s2.getPavarde() == s1.getPavarde());
00280     testPassed("Round-trip - pavarde saugai issaugotas ir nuskaitytas");
00281
00282     assert(s2.getPaz() == s1.getPaz());
00283     testPassed("Round-trip - pazymiai saugai issaugoti ir nuskaityti");
00284 }
00285
00286 // TEST #11: Modifikavimas po kopijavimo
00287 void test_modification_after_copy() {
00288     printTestHeader("Modifikavimas po kopijavimo");
00289
00290     Studentas s1("Elena", "Elene", 85);
00291     s1.addPaz(7);
00292     s1.addPaz(8);
00293
00294     Studentas s2 = s1;
00295
00296     s2.setEgz(95);
00297     assert(s1.getEgz() == 85 && s2.getEgz() == 95);
00298     testPassed("Egzamino modifikacija - originalas nepaveiktas");
00299
00300     s2.addPaz(9);
00301     assert(s1.getPaz().size() == 2 && s2.getPaz().size() == 3);
00302     testPassed("Pazymio pridejimas - originalas nepaveiktas");
00303 }
00304
00305 // TEST #12: Getter/Setter operaciju konsekvencija
00306 void test_getters_setters() {
00307     printTestHeader("Getters/Setters Konsekvencija");
00308
00309     Studentas s;
00310
00311     s.setVardas("Tomas");
00312     assert(s.getVardas() == "Tomas");
00313     testPassed("Vardas setter/getter veikia");
00314
00315     s.setPavarde("Tomas");
00316     assert(s.getPavarde() == "Tomas");
00317     testPassed("Pavarde setter/getter veikia");
00318
00319     s.setEgz(100);
00320     assert(s.getEgz() == 100);
00321     testPassed("Egzaminas setter/getter veikia");
00322
00323     s.setRez(90.5);
00324     assert(s.getRez() == 90.5);
00325     testPassed("Rezultatas setter/getter veikia");
00326
00327     s.setMed(88.0);
00328     assert(s.getMed() == 88.0);
00329     testPassed("Mediana setter/getter veikia");
00330 }
00331
00332 // TEST #13: Vector operacijos su pazymiais
00333 void test_paz_operations() {
00334     printTestHeader("Vector Operacijos su Pazymiais");
00335
00336     Studentas s;
00337
00338     assert(s.isPazEmpty());
00339     testPassed("isPazEmpty() grazina true tusciam sarasui");
00340
00341     s.addPaz(10);
00342     s.addPaz(9);
00343     assert(!s.isPazEmpty());
00344     testPassed("isPazEmpty() grazina false ne tusciam sarasui");
00345

```

```

00346     assert(s.getPaz().size() == 2);
00347     testPassed("Pazymiai pridedami teisingai");
00348
00349     s.clearPaz();
00350     assert(s.isPazEmpty());
00351     testPassed("clearPaz() isvalo pazymius");
00352 }
00353
00354 // ===== PAPILDOMI GOOGLE TEST TESTAI (v2.0) =====
00355
00356 // TEST #14: Vektoriaus su Studentais operacijos
00357 TEST(StudentasVectorTest, PushBackCopySemantics) {
00358     vector<Studentas> students;
00359
00360     Studentas s1("Audra", "Audre", 91);
00361     s1.addPaz(9);
00362     s1.addPaz(8);
00363
00364     students.push_back(s1); // Copy semantika
00365
00366     EXPECT_EQ(students.size(), 1);
00367     EXPECT_EQ(students[0].getVardas(), "Audra");
00368     EXPECT_EQ(students[0].getPaz().size(), 2);
00369 }
00370
00371 // TEST #15: Move semantika su vektoriumi
00372 TEST(StudentasVectorTest, PushBackMoveSemantics) {
00373     vector<Studentas> students;
00374
00375     Studentas s1("Giedra", "Giedre", 88);
00376     s1.addPaz(8);
00377
00378     students.push_back(std::move(s1)); // Move semantika
00379
00380     EXPECT_EQ(students.size(), 1);
00381     EXPECT_TRUE(s1.isPazEmpty()); // Originalas turi būti tuščias
00382     EXPECT_EQ(students[0].getVardas(), "Giedra");
00383 }
00384
00385 // TEST #16: Deep copy - independenti duplikacija
00386 TEST(StudentasDeepCopyTest, IndependentCopies) {
00387     Studentas original("Laimas", "Laimas", 92);
00388     original.addPaz(9);
00389     original.addPaz(10);
00390
00391     Studentas copy1 = original;
00392     Studentas copy2 = original;
00393
00394     // Modifikuojame copy1
00395     copy1.setEgz(100);
00396     copy1.addPaz(8);
00397
00398     // Patikrinti, kad originalas ir copy2 nepakeisti
00399     EXPECT_EQ(original.getEgz(), 92);
00400     EXPECT_EQ(original.getPaz().size(), 2);
00401     EXPECT_EQ(copy2.getEgz(), 92);
00402     EXPECT_EQ(copy2.getPaz().size(), 2);
00403     EXPECT_EQ(copy1.getEgz(), 100);
00404     EXPECT_EQ(copy1.getPaz().size(), 3);
00405 }
00406
00407 // TEST #17: Self-assignment saugumas
00408 TEST(StudentasSelfAssignmentTest, CopySafetyCheck) {
00409     Studentas s("Monika", "Monika", 95);
00410     s.addPaz(10);
00411     s.addPaz(9);
00412
00413     // Self-assignment - turi būti saugus
00414     s = s;
00415
00416     EXPECT_EQ(s.getVardas(), "Monika");
00417     EXPECT_EQ(s.getEgz(), 95);
00418     EXPECT_EQ(s.getPaz().size(), 2);
00419 }
00420
00421 // TEST #18: Operatoriaus lankstumo - chaining
00422 TEST(StudentasOperatorTest, AssignmentChaining) {
00423     Studentas s1("Vincas", "Vincas", 92);
00424     Studentas s2("Petras", "Petrauskas", 88);
00425     Studentas s3;
00426
00427     // Assignment chaining: (s3 = s2) = s1;
00428     // Step 1: s3 = s2 grąžina s3 referencę, s3 becomes "Petras"
00429     // Step 2: (s3 referencija) = s1 priskyrimo s1 reikšmę s3
00430     (s3 = s2) = s1;
00431
00432     EXPECT_EQ(s3.getVardas(), "Vincas"); // s3 final value is Vincas

```

```
00433     EXPECT_EQ(s2.getVardas(), "Petras"); // s2 remains Petras
00434     EXPECT_EQ(s1.getVardas(), "Vincas"); // s1 remains Vincas
00435 }
00436
00437 // ===== ORIGINAL MAIN (su backward compatibility) =====
00438 int main(int argc, char** argv) {
00439     // Google Test inicializacija
00440     ::testing::InitGoogleTest(&argc, argv);
00441
00442     // Originalūs testai (v1.5)
00443     cout << "\n" << string(60, '*') << endl;
00444     cout << "    STUDENTAS KLASĖS VISOS METODŲ TESTAI (v1.5)" << endl;
00445     cout << "    + GOOGLE TEST PAPILDYMAS (v2.0)" << endl;
00446     cout << string(60, '*') << endl;
00447
00448     try {
00449         test_default_constructor();
00450         test_parametrized_constructor();
00451         test_copy_constructor();
00452         test_copy_assignment();
00453         test_move_constructor();
00454         test_move_assignment();
00455         test_destructor();
00456         test_output_operator();
00457         test_input_operator();
00458         test_io_roundtrip();
00459         test_modification_after_copy();
00460         test_getters_setters();
00461         test_paz_operations();
00462
00463         cout << "\n" << string(60, '*') << endl;
00464         cout << "TESTO REZULTATAI:" << endl;
00465         cout << "Priimti testai: " << testai_praletii << endl;
00466         cout << "Nepriimti testai: " << testai_nepraletii << endl;
00467         cout << string(60, '*') << endl;
00468
00469         if (testai_nepraletii == 0) {
00470             cout << "\nVISI TESTAI PRIIMTI!" << endl;
00471         } else {
00472             cout << "\nKai kurie testai nepraejo!" << endl;
00473             return 1;
00474         }
00475     }
00476     catch (const std::exception& e) {
00477         cout << "Kritinė klaida: " << e.what() << endl;
00478         return 1;
00479     }
00480
00481     cout << "\n" << string(60, '*') << endl;
00482     cout << "Vykdome GOOGLE TEST testai..." << endl;
00483     cout << string(60, '*') << "\n" << endl;
00484
00485     // Vykdyti Google Test testus
00486     return RUN_ALL_TESTS();
00487 }
```


Rodyklė

~Studentas
 Studentas, [16](#)
~Zmogus
 Zmogus, [22](#)
addPaz
 Studentas, [17](#)

clearPaz
 Studentas, [17](#)

deque< T >, [9](#)
deque< T >::const_iterator, [7](#)
deque< T >::const_reverse_iterator, [8](#)
deque< T >::iterator, [10](#)
deque< T >::reverse_iterator, [13](#)

exception, [9](#)
extra_cpp/duomenu_valdymas.cpp, [25](#)
extra_cpp/input.cpp, [30](#)
extra_cpp/mat_funkcijos.cpp, [31](#)
extra_cpp/menu.cpp, [32](#)
extra_cpp/output.cpp, [33](#)
extra_cpp/Studentas.cpp, [34](#)
extra_cpp/testavimas.cpp, [36](#)
extra_cpp/Zmogus.cpp, [41](#)

getEgz
 Studentas, [17](#)
getInfo
 Studentas, [17](#)
 Zmogus, [22](#)
getMed
 Studentas, [17](#)
getPavarde
 Zmogus, [22](#)
getPaz
 Studentas, [18](#)
getRez
 Studentas, [18](#)
getVarDas
 Zmogus, [23](#)

header_files/duomenu_valdymas.h, [41](#)
header_files/input.h, [41](#)
header_files/Mat_funkcijos.h, [42](#)
header_files/menu.h, [42](#)
header_files/output.h, [42](#)
header_files/Studentas.h, [42](#)
header_files/testavimas.h, [43](#)

header_files/Zmogus.h, [43](#)

ifstream, [10](#)
invalid_argument, [10](#)
isPazEmpty
 Studentas, [18](#)
istream, [10](#)

list< T >, [11](#)
list< T >::const_iterator, [7](#)
list< T >::const_reverse_iterator, [8](#)
list< T >::iterator, [11](#)
list< T >::reverse_iterator, [13](#)

ofstream, [12](#)
operator<<
 Studentas, [19](#)
 Zmogus, [24](#)
operator>>
 Studentas, [19](#)
operator=
 Studentas, [18](#)
ostream, [12](#)

pavarde_
 Zmogus, [24](#)

readStudent
 Studentas, [18](#)
reservePaz
 Studentas, [19](#)
runtime_error, [14](#)

setEgz
 Studentas, [19](#)
setMed
 Studentas, [19](#)
setPavarde
 Zmogus, [23](#)
setPaz
 Studentas, [19](#)
setRez
 Studentas, [19](#)
setVarDas
 Zmogus, [23](#)
string, [14](#)
string::const_iterator, [7](#)
string::const_reverse_iterator, [8](#)
string::iterator, [11](#)
string::reverse_iterator, [13](#)

stringstream, [14](#)

Studentas, [15](#)

 ~Studentas, [16](#)

 addPaz, [17](#)

 clearPaz, [17](#)

 getEgz, [17](#)

 getInfo, [17](#)

 getMed, [17](#)

 getPaz, [18](#)

 getRez, [18](#)

 isPazEmpty, [18](#)

 operator<<, [19](#)

 operator>>, [19](#)

 operator=, [18](#)

 readStudent, [18](#)

 reservePaz, [19](#)

 setEgz, [19](#)

 setMed, [19](#)

 setPaz, [19](#)

 setRez, [19](#)

 Studentas, [16](#), [17](#)

vardas_

 Zmogus, [24](#)

vector< T >, [20](#)

vector< T >::const_iterator, [8](#)

vector< T >::const_reverse_iterator, [9](#)

vector< T >::iterator, [11](#)

vector< T >::reverse_iterator, [13](#)

Zmogus, [20](#)

 ~Zmogus, [22](#)

 getInfo, [22](#)

 getPavarde, [22](#)

 getVardas, [23](#)

 operator<<, [24](#)

 pavarde_, [24](#)

 setPavarde, [23](#)

 setVardas, [23](#)

 vardas_, [24](#)

 Zmogus, [22](#)